



## **Project Report**

On

**Brain2Text**

**Neural Signal Decoding to Text via GRU-based**

**Deep Learning and Web Deployment**

**Submitted to D Y Patil International University, Akurdi, Pune  
in partial fulfilment of full-time degree**

Bachelor of Technology

**Submitted By:**

Sidheshwar Sharma PRN:20220802331

Aayush Chougule PRN:20220802333

Uday Tomar PRN:20220802412

Under the Guidance of

**Dr. Gaurav Kumar Singh**

Sr. Assistant Professor

School of Computer Science, Engineering and Applications

**D Y Patil International University, Akurdi, Pune, INDIA - 411044**

[Session 2025-26]



**D Y PATIL**  
**INTERNATIONAL**  
**UNIVERSITY**  
AKURDI PUNE

## **CERTIFICATE**

This is to certify that Sidheshwar Sharma (PRN:20220802331), Aayush Chougule (PRN:20220802333), Uday Tomar (PRN:20220802412) are bonafide students of the School of Computer Science Engineering and Applications (SCSEA), have satisfactorily completed the project work entitled “Brain2Text” submitted to D Y Patil International University, Pune in partial fulfillment for the award of Bachelor of Technology- CSE ( AI/ML and DS ) in the academic year 2025-26. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated into the report deposited in SCSEA. The project report has been approved as it satisfies the academic requirements in respect of project report prescribed for the said degree.

**Dr. Gaurav Kumar Singh**  
Project Guide

**Dr. Sanjay Mohite**  
Project Coordinator

**Prof. (Dr.) Rahul Sharma**

Director

School of Computer Science Engineering & Applications

D Y Patil International University, Akurdi

Pune, Maharashtra, India- 411044

# Declaration

---

We hereby declare that the project entitled “**Brain2Text**” : ”Neural Signal Decoding to Text via GRU-based Deep Learning and Web Deployment” submitted by us is original and the research work has been carried out independently at the School of Computer Science Engineering and Applications, under the guidance of Dr. Gaurav Kumar Singh. This report has been submitted in partial fulfillment for the award of the degree of Bachelor of Technology. We also declare that the matter embodied in this report has not been submitted by us for the award of any other degree of any other University or Institute.

**Name:** Sidheshwar Sharma

**PRN:** 20220802331

**Name:** Aayush Chougule

**PRN:** 20220802333

**Name:** Uday Tomar

**PRN:** 20220802412

# Acknowledgment

---

We extend our deep sense of gratitude to our respected guide Dr. Gaurav Kumar Singh, for his valuable help and guidance. We are thankful for the encouragement that he has given us in completing this project successfully. It is imperative for us to mention the fact that the report of project could not have been accomplished without the periodic suggestions and advice of our project guide.

We are grateful to Dr. Sanjay Mohite, Project Coordinator, and to Prof. (Dr.) Rahul Sharma, Director, SCSEA, for their academic oversight and for providing us with access to the computational resources required to train and evaluate our neural decoding models.

With due respect, we express our profound gratitude to our Hon'ble Vice Chancellor, DYPIU, Akurdi, Prof. (Dr.) Manish Bhalla, for his visionary leadership and unwavering support, which have been instrumental in the successful completion of this project. We are truly honored to have had access to the exemplary facilities and resources of the institution under his esteemed guidance.

Our sincere thanks go to the open-source community behind PyTorch, Flask, React, and Vite, whose well-documented frameworks made the full-stack implementation of our system possible within the project timeline.

We acknowledge the organisers of the Brain-to-Text Challenge 2025 on Kaggle for making the T15 CopyTask dataset publicly available. This dataset formed the empirical backbone of our work and without it the project would have remained theoretical.

Finally, we thank our families and classmates for their unwavering moral support during the longer debugging sessions that this project inevitably demanded.

**Name:** Sidheshwar Sharma

**PRN:** 20220802331

**Name:** Aayush Chougule

**PRN:** 20220802333

**Name:** Uday Tomar

**PRN:** 20220802412

## Abstract

---

We're used to thinking about communication in terms of talk, and conversation as a normal part of our lives. For people with disabilities like A.L.S., locked in or severe motor impairments, this communication ability can be forever lost. Over the last 20 years, researchers in the field of brain-computer interface (BCI) have been trying to find a solution: decode the brain signals that are involved when people try to speak, turn them into meaningful text, and read it on screen! Brain2Text is such a pipeline – a machine learning-component, a signal processing component, and a state-of-the-art web-component tipping them together and making it deployable as a system – this project is a Final Year Engineering Project.

Brain-to-Text Challenge 2025 Benchmark is used, which was created by Kaggle based on an experimental paradigm named T15 CopyTask. These are intracortical recordings from a speech motor cortex area in an ararthric subject in a 256 electrode Utah array. During each trial the signals are recorded while the subject tries to speak a phrase and pre-processed into feature vectors of dimension 512 and length of N time-steps (N varies based on the task). In addition to the ground truth transcriptions, the dataset provides phoneme and word level transcriptions and neural recordings which are well suited to be used in a sequence-to-sequence decoding approach.

Brain2Text system is trained using a method known as the Connectionist Temporal Classification (CTC) loss and based on a Gated Recurrent Unit (GRU) network. Its architecture consists of a linear adapter layer that reduces the 512 dimensional neural input to a 256 dimensional embedding by reducing the channel dimension followed by five unidirectional GRUs with a hidden number of units of size 768. A fully connected output layer is used to project the Gru output onto 40 ARPAbet phonemes plus a special CTC blank token and log-softmax as an activation function generates the probability distribution of the output sample which is utilized in a greedy decoding procedure during inference. The weights of the model's pre-trained version can be stored with the project in a checkpoint form (file extension .pkl) and are included with the project, so that the model can be loaded and applied without having to set up training infrastructure.

The inference module takes in the raw output of the model: a sequence of per-frame phoneme probabilities and uses a two-stage decoder to output a sentence that is readable by humans. The first step is a greedy CTC decoding, in which the frame level predictions are reduced to a string of phonemes separated by spaces and word boundaries by pipes. In the second stage, a phoneme-to-text module does an attempt to search from the CMU Pronouncing Dictionary,

which ranks the word by the frequency of occurrence in the Brown corpus, for each word boundary group and can find the most frequent one more likely than the phonetic, but less frequent alternative among the same words. If the word in the dictionary is missing for a phoneme group, then, there is a “fallback” to the Pollinations API with an LLM system generating a reasonable answer.

The web app has 5 separate endpoints: Three endpoints for the Flask REST API: health check; decode and predict the text for a live input phoned sequence; decode and predict the text from a batch of files containing multipart .npy files; decode/transcribe a single phoned sequence to text; A demo endpoint with a list of some examples of phoned sequences to decode and predict. This backend is used by a React and Vite frontend, which allows the user to upload a .npy neural recording and get back a decoded phrase, or to simulate the complete pipeline locally, with eight pre-recorded neural recordings of typical phrases (such as “hello world”, “good morning”, “thank you”, and “open the door”). They both provide animated EEG like images of the signals and typed text/visualised text output to enable text interpretation by a non-specialist audience.

The evaluation of system was done not numerically but functionally; emphasis was on how the application functions on the deployable system. All 8 demo trials were decoded into their respective phrases and latencies were consistently under 200 ms on CPU hardware, even when all the request types were tested, the Flask API returned all well formed JSON responses. By applying a linguistically motivated phoneme-based decoder, they were able to deal with text unpredictability, whereas by using an off-the-shelf GRU backbone, they were also able to deal with the variability of words across languages, deriving together coherent short sentences from raw neural data and without the need for fine-tuning on the deployment machine.

They show that, to turn an intra-cortical imaging signal acquisition system into a final imaging signal output system on the browser interface, only a few known and easily available open source software tools are needed to be implemented within the time frame typical of an undergraduate student project. The disadvantages are the limited vocabulary of the CTC model, the lack of an LM beam search decoder and the requirement of clean pre-processed feature vectors in order to not have to accept raw spike trains. Future directions involve the implementation of a Transformer based acoustic model, improvement using online streaming, extending sets of phoenemes for non-English languages..

*Keywords: brain-computer interface, neural decoding, GRU, CTC, ARPAbet, phoneme, Flask, React, intracortical recording*

# Table of Contents

|                                                                              |             |
|------------------------------------------------------------------------------|-------------|
| <b>DECLARATION</b>                                                           | <b>i</b>    |
| <b>ACKNOWLEDGEMENT</b>                                                       | <b>ii</b>   |
| <b>ABSTRACT</b>                                                              | <b>iii</b>  |
| <b>LIST OF FIGURES</b>                                                       | <b>viii</b> |
| <b>LIST OF TABLES</b>                                                        | <b>ix</b>   |
| <b>LIST OF ABBREVIATIONS</b>                                                 | <b>x</b>    |
| <b>1 Introduction</b>                                                        | <b>1</b>    |
| 1.1 Problem Statement . . . . .                                              | 1           |
| 1.2 Objectives . . . . .                                                     | 2           |
| 1.3 Purpose . . . . .                                                        | 2           |
| 1.4 Scope . . . . .                                                          | 2           |
| 1.5 Applicability . . . . .                                                  | 3           |
| <b>2 Literature Survey</b>                                                   | <b>4</b>    |
| 2.1 Literature Survey . . . . .                                              | 4           |
| 2.1.1 Intracortical Recording and Motor Cortex Decoding . . . . .            | 4           |
| 2.1.2 EEG-based Approaches . . . . .                                         | 4           |
| 2.1.3 Recurrent and Transformer Architectures . . . . .                      | 5           |
| 2.1.4 Sequence-to-Sequence Neural Decoding . . . . .                         | 5           |
| 2.2 Comparison Table of Related Works . . . . .                              | 6           |
| 2.3 Gap Analysis . . . . .                                                   | 6           |
| <b>3 Existing Systems and Problem Context</b>                                | <b>8</b>    |
| 3.1 Traditional Augmentative and Alternative Communication Devices . . . . . | 8           |
| 3.2 Existing BCI Approaches . . . . .                                        | 8           |
| 3.3 Limitations of Existing Systems . . . . .                                | 9           |
| 3.4 Proposed System Overview . . . . .                                       | 10          |
| <b>4 GRU Architecture and Neural Network Design</b>                          | <b>11</b>   |
| 4.1 Why GRU Over Other Recurrent Architectures . . . . .                     | 11          |
| 4.2 GRU Gating Mechanism . . . . .                                           | 11          |
| 4.3 Stacked GRU Layers . . . . .                                             | 12          |
| 4.4 Linear Adapter Layer . . . . .                                           | 12          |
| 4.5 Output Projection and CTC Loss . . . . .                                 | 13          |
| 4.6 Parameter Count . . . . .                                                | 13          |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.7      | ARPAbet Phoneme Representation . . . . .                       | 14        |
| <b>5</b> | <b>Training, Hyperparameter Configuration and Optimisation</b> | <b>15</b> |
| 5.1      | Training Environment . . . . .                                 | 15        |
| 5.2      | Adam Optimiser Configuration . . . . .                         | 15        |
| 5.3      | Batch Construction and Padding . . . . .                       | 16        |
| 5.4      | CTC Loss Formulation . . . . .                                 | 16        |
| <b>6</b> | <b>Proposed Methodology</b>                                    | <b>17</b> |
| 6.1      | System Architecture . . . . .                                  | 17        |
| 6.2      | Proposed Methodology . . . . .                                 | 18        |
| 6.2.1    | Data Acquisition and Dataset Description . . . . .             | 18        |
| 6.2.2    | Data Pre-processing . . . . .                                  | 19        |
| 6.2.3    | Model Architecture . . . . .                                   | 19        |
| 6.2.4    | Training Methodology . . . . .                                 | 20        |
| 6.2.5    | Inference Pipeline . . . . .                                   | 21        |
| 6.2.6    | Phoneme-to-Text Decoding . . . . .                             | 22        |
| 6.2.7    | Backend Architecture . . . . .                                 | 22        |
| 6.2.8    | Frontend Architecture . . . . .                                | 23        |
| 6.2.9    | Deployment . . . . .                                           | 24        |
| 6.3      | Hardware and Software Requirements . . . . .                   | 25        |
| <b>7</b> | <b>Implementation Details</b>                                  | <b>26</b> |
| 7.1      | Project Directory Structure . . . . .                          | 26        |
| 7.2      | Model Loading and Checkpoint Format . . . . .                  | 26        |
| 7.3      | CTC Greedy Decoding Implementation . . . . .                   | 27        |
| 7.4      | Flask Application Design . . . . .                             | 28        |
| 7.5      | React Frontend Implementation . . . . .                        | 28        |
| 7.6      | Phoneme-to-Text Module . . . . .                               | 29        |
| 7.7      | Dependency Setup and Environment Issues . . . . .              | 29        |
| 7.8      | Testing Approach . . . . .                                     | 30        |
| <b>8</b> | <b>Result and Discussion</b>                                   | <b>32</b> |
| 8.1      | System Functionality and Evaluation Approach . . . . .         | 32        |
| 8.2      | Model Performance . . . . .                                    | 32        |
| 8.3      | Demo Trial Results . . . . .                                   | 32        |
| 8.4      | API Response Behaviour . . . . .                               | 33        |
| 8.5      | Frontend Behaviour . . . . .                                   | 34        |
| 8.6      | Strengths and Limitations . . . . .                            | 34        |



|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Performance Analysis and Error Discussion</b>           | <b>36</b> |
| 9.1       | Observed Decoding Errors and Their Sources . . . . .       | 36        |
| 9.2       | Latency Breakdown . . . . .                                | 36        |
| 9.3       | Comparative Performance Context . . . . .                  | 38        |
| 9.4       | Real-World Application Scenarios . . . . .                 | 38        |
| 9.5       | Identified Engineering Challenges . . . . .                | 39        |
| <b>10</b> | <b>Real-World Applications and Broader Impact</b>          | <b>40</b> |
| 10.1      | Assistive Technology for Communication Disorders . . . . . | 40        |
| 10.2      | Neuroprosthetics and Motor Control . . . . .               | 40        |
| 10.3      | Cognitive Neuroscience Research . . . . .                  | 41        |
| 10.4      | Human-Computer Interaction Without Motor Input . . . . .   | 41        |
| 10.5      | Educational and Outreach Value . . . . .                   | 41        |
| 10.6      | Integration with Larger Language Models . . . . .          | 42        |
| <b>11</b> | <b>Conclusion and Future Scope</b>                         | <b>43</b> |
| 11.1      | Conclusion . . . . .                                       | 43        |
| 11.2      | Future Scope . . . . .                                     | 43        |
|           | <b>REFERENCES</b>                                          | <b>45</b> |
| <b>A</b>  | <b>Appendix</b>                                            | <b>47</b> |
| A.1       | Project Repository Structure . . . . .                     | 47        |
| A.2       | Python Dependency Versions . . . . .                       | 48        |
| A.3       | ARPAbet Phoneme Vocabulary . . . . .                       | 48        |
| A.4       | Flask API Request and Response Formats . . . . .           | 48        |
| A.5       | Demo Trial Phoneme Reference . . . . .                     | 50        |
| A.6       | Startup Script Reference . . . . .                         | 50        |

## List of Figures

|     |                                                                                                                                                  |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Comparison between the typical research-only workflow (left) and the Brain2Text deployable approach (right). . . . .                             | 10 |
| 6.1 | Overall Brain2Text system architecture, showing the three-layer design from neural input to browser output. . . . .                              | 17 |
| 6.2 | Data pre-processing flowchart, from file loading to batch-ready tensor. . . . .                                                                  | 19 |
| 6.3 | Model training pipeline, from dataset batching through CTC loss to checkpoint saving. . . . .                                                    | 21 |
| 6.4 | Backend–frontend interaction diagram showing the full request cycle from file upload to decoded text. . . . .                                    | 23 |
| 6.5 | Deployment architecture: local two-process launch via <code>start.sh</code> (top) and Colab-hosted deployment via ngrok tunnel (bottom). . . . . | 24 |

## List of Tables

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 2.1 | Comparison of representative neural speech decoding studies . . . . .       | 6  |
| 4.1 | Approximate trainable parameter count by model component . . . . .          | 14 |
| 6.1 | Brain-to-Text 2025 T15 CopyTask Dataset Description . . . . .               | 18 |
| 6.2 | GRU Model Hyperparameters . . . . .                                         | 20 |
| 6.3 | Flask API Endpoints . . . . .                                               | 23 |
| 6.4 | Hardware Requirements . . . . .                                             | 25 |
| 6.5 | Software Requirements . . . . .                                             | 25 |
| 7.1 | Summary of Component Testing Results . . . . .                              | 31 |
| 8.1 | Demo trial expected outputs and observed decoding behaviour . . . . .       | 33 |
| 9.1 | Measured end-to-end latency breakdown (live mode, 100-step trial) . . . . . | 37 |
| 9.2 | Performance comparison across neural decoding systems . . . . .             | 38 |
| A.1 | Python dependencies (requirements.txt) . . . . .                            | 48 |
| A.2 | ARPAbet phoneme vocabulary (index 0 reserved for CTC blank) . . . . .       | 48 |
| A.3 | Complete demo trial phoneme reference . . . . .                             | 50 |

# LIST OF ABBREVIATIONS

|                |                                                                        |
|----------------|------------------------------------------------------------------------|
| <b>AI</b>      | Artificial Intelligence                                                |
| <b>API</b>     | Application Programming Interface                                      |
| <b>ARPAbet</b> | A Phonetic Alphabet developed by the Advanced Research Projects Agency |
| <b>BCI</b>     | Brain-Computer Interface                                               |
| <b>CMU</b>     | Carnegie Mellon University                                             |
| <b>CPU</b>     | Central Processing Unit                                                |
| <b>CTC</b>     | Connectionist Temporal Classification                                  |
| <b>DL</b>      | Deep Learning                                                          |
| <b>ECoG</b>    | Electrocorticography                                                   |
| <b>EEG</b>     | Electroencephalography                                                 |
| <b>FC</b>      | Fully Connected (layer)                                                |
| <b>GPU</b>     | Graphics Processing Unit                                               |
| <b>GRU</b>     | Gated Recurrent Unit                                                   |
| <b>HTTP</b>    | Hypertext Transfer Protocol                                            |
| <b>JSON</b>    | JavaScript Object Notation                                             |
| <b>LLM</b>     | Large Language Model                                                   |
| <b>LSTM</b>    | Long Short-Term Memory                                                 |
| <b>ML</b>      | Machine Learning                                                       |
| <b>NLP</b>     | Natural Language Processing                                            |
| <b>np</b>      | NumPy Binary Array Format                                              |
| <b>PER</b>     | Phoneme Error Rate                                                     |
| <b>pkl</b>     | Python Pickle Serialisation Format                                     |
| <b>RNN</b>     | Recurrent Neural Network                                               |
| <b>REST</b>    | Representational State Transfer                                        |
| <b>STT</b>     | Speech-to-Text                                                         |
| <b>UI</b>      | User Interface                                                         |
| <b>WER</b>     | Word Error Rate                                                        |

# 1 Introduction

Communication is a product of a brain process. The motor cortex has already formulated, ordered and started motor programme that will produce the one syllable that was put forward before a single sound is made. People who lost voluntary speech due to neurological damage and/or degeneration have this intention and/or planning stage, but there is damage to the motor pathway from the intention to the vocal apparatus. The brain speaks to the world by using a new computer technology that translates the brain's signals into speech.

There have been relatively consistent gains in this area in the last ten years. Willett et al. (2023) [1] introduced an intracortical decoding algorithm for attempted hand-writing at a fundamental character level (90 characters per minute) and Moses et al. (2021) developed a method for restoring sentence-level communication to a patient paralyzed by decoding sub-vocal speech attempts using electrocorticographic (ECoG) signals from their brain. Recently, Metzger et al. [2] have demonstrated more than 60 words per minute with the combination of an intracortical decoder and a large language model post processor on the 125,000-word data set. This work was extended to create a standardised benchmark dataset, known as the T15 CopyTask, on Kaggle, which can be used to repeatably compare different decoding architectures for the Brain-to-Text Challenge 2025. This project, called Brain2Text, aims to support this research path by creating a fully functional deployable system based on the challenges bottom-up approach.

## 1.1 Problem Statement

The present day relies on devices for augmentative and alternative communication (AAC) such as eye trackers, sip-and-puff switches, or a head-mounted pointer with a very limited use of motor control and are even slower than natural speech. There would be more of a "brain to text" interface than a "brain to peripheral motor system" interface, so the speed would be closer to natural speech frequencies, and the physical size of the peripheral motor system would be reduced significantly. The basic technical issue is the following: in order to understand the meaning of a word or a sequence of phonemes in the context of a spoken sequence, how can a mapping be found between a sequence of high dimensional neural feature vectors, observed as a person attempts to say a word, or sequence of phonemes, and the corresponding sequence of phonemes or words?

Specifically, input into the Brain2Text decoding system is a two-dimensional input NumPy array of shape  $[T, 512]$ , where  $T$  is the length of a trial (usually filtered to 75–200 frames at the temporal resolution of the dataset) and 512 is the number of features extracted from the Utah

array before processing. This system is supposed to give the answer as the text string with the text to be said by the participant.

## 1.2 Objectives

The project sets out to achieve the following:

- a) Integrate a pre-trained GRU-based sequence model on the Brain-to-Text 2025 dataset to decode ARPAbet phoneme sequences from neural feature inputs.
- b) Implement a two-stage phoneme-to-text decoder combining CMU dictionary lookup with frequency-weighted word selection and an LLM fallback for unresolved phoneme groups.
- c) Build a Flask REST API that wraps the inference pipeline and exposes clean HTTP endpoints for real-time and batch decoding.
- d) Develop a React-based frontend with live and demo operating modes, animated signal visualisation, and typed-text output rendering.
- e) Package the complete system with a startup script (`start.sh`) and a Google Colab deployment path for environments without a local GPU.

## 1.3 Purpose

Brain2Text has two uses. First of all, this is a proof-of-concept that shows a functioning neural speech decoder can be wrapped into a webapp that everyone can access using any modern browser, without relying on the end user to install python modules or to take care of the model weights. Second, it details the design and implementation decisions and implications of hardware choices made for a GRU-CTC decoder applied to the T15 CopyTask benchmark for future students or researchers to extend, for building more capable machines.

## 1.4 Scope

This project is a "bounded" project. Signal processing aspects—the system is not required to extract features from the electrophysiological spike trains, but uses pre-extracted 512-dimensional feature vectors (which are a black box given by the challenge organisers). Vocabulary is restricted to 40 ARPAbet phonemes for modelling, so that it can only decode

short sentences in the English language. On the deployment side, it's not about cloud-scaled production hosting, but about executing it locally or on Colab.

### 1.5 Applicability

This more advanced technology is immediately used by those with anarthria, ALS, brainstem stroke or locked-in syndrome (a stroke in the area of the brain that connects the brain to the body). In addition to AT, the decoding pipeline here is relevant to cognitive neuroscience research (research of speech planning in the motor cortex), neuroprosthetics (speech imagined and control of external devices) and human-computer interaction (HCI) research (external devices controlled by means of speech, studied are input modalities other than muscular ones).

Brain2Text is an example of the growing trend in applied AI of taking an available pre-trained model from a public benchmark, wrapping it in a REST API, placing it in a browser app, and deploying it in a single command on the server. The particular domain is new; the software architecture (that is, what the student has to build) is very familiar so that the project can be used as a guide in the future for other projects comparing other neural decoding modalities.

## 2 Literature Survey

### 2.1 Literature Survey

Research into neural speech decoding has accelerated substantially since the mid-2010s, driven by improvements in electrode technology, advances in deep sequence models, and the release of large annotated neural datasets. This section surveys ten representative works that shaped the methods used in Brain2Text, grouping them by the modality of neural recording and the decoding approach employed.

#### 2.1.1 Intracortical Recording and Motor Cortex Decoding

Willett et al. [1] have measured intracortical signals from a human with ALS with a Utah array in the hand-knob region of the motor cortex. The participants tried to write by taking single letter at a time and a recurrent neural network successfully read out the letters with 94.1

Recently, Anumanchipalli et al. [3] showed that the EEG signals recorded from the speech motor cortex could be used to synthesize comprehensible utterances. In the system they used a two stage neural network: Decoding (which is converting cortical activities to articulatory gestures) and Synthesis (which is converting audio from the gestures). The output modality is not identical to our text-based approach but the free citation of the neural signals to the communicable message spurred us to use ARPAbet as intermediate decoding target.

Moses et al. [4] concentrated on a brainstem stroke patient with near total paralysis, but no cognitive impairments. They used their system with CTC loss to decode 15-WPM 50-word vocabulary sentences using an RNN. The goal of Brain2Text training was directly adopted to the explicit use of CTC for training neural sequence decoding with no need of frame-level annotations for this task.

#### 2.1.2 EEG-based Approaches

However, Nakanishi et al. [5] used a deep learning method (CNN) to classify above chance a collection of individual words by using scalp EEG signals that were recorded when words from a small vocabulary were imagined, thereby demonstrating the successful classification of imagined speech words. Even though the recording are non-invasive and no surgery is needed, methods based on EEG have a much lower spatial resolution than intracortical recording, so that the phoneme level detail extracted from the recording is much less.



Wang et al. [6] extended the decoding into 40 individual words with a fine-tuned BERT encoder and spatial filtering taking advantage of the word-to-word connection. Their best word classification accuracy was 53% on the 40-class problem, which is quite far off intracortical level accuracy, but indicates the potential of vocabulary-sized EEG decoding.

### **2.1.3 Recurrent and Transformer Architectures**

In Makin et al. [7] a vocabulary of 250 words was used for recording ECoG and an audio processor was used to transcribe it; their test word error rate was 3 percent. They applied an encoder-decoder LSTM with attention, as a result they did an example of neural decoding. The example they shared was the neural decoding with encoder-decoder LSTM with attention; this was a template for future examples like the baseline notebook for the Brain-to-Text 2025 challenge.

Défossez et al. [8] used a gigantic pre-trained speech representation model (wav2vec 2.0) to decode perceived speech from MEG and/or EEG recordings. The method they used didn't require task-specific training to gather data but instead required the participant to hear speech to use the method, allowing it to be adapted to expressive speech prosthetics. In future releases of the Brain2Text, however, it would be interesting to expand on the use of pre-trained speech features as the alignment targets.

Tang et al. [9] used a Transformer-based model to partition continuous natural speech extracted from non-invasive fMRI to reconstruct the semantic contents of the sentences, achieving a good performance. The study also differs in methodology from our study, as it used fMRI over intracortical, perception over production, word-level over phoneme-level, but it was found that sequential neural decoding with Transformers is now more competitive than RNN based approaches.

### **2.1.4 Sequence-to-Sequence Neural Decoding**

Herff et al. [10] employed an LSTM network on ECoG data to generate continuous speech synthesiser parameters, and were able to reconstruct intelligible speech in near real-time. Their work was not at the phoneme level, but rather at the articulatory feature level, and their use of LSTM for temporal modelling of speech motor signals led to a change of the baseline architecture of the Brain-to-Text challenge to using GRU instead of LSTM for greater training stability.

Metzger et al. [2] reported a speech BCI with more than 60 words per minute on a vocabulary of 125,000 words by using intracortical recordings and a fine-tuned large language model post-

processor. Their approach to reranking the outputs of the phoneme decoder with a language model is similar to what we do in our LLM fallback step in `phoneme_to_text.py`, but our implementation is more simple and intended to be deployed without the extra GPU inference cost.

## 2.2 Comparison Table of Related Works

Table 2.1 summarises the key characteristics and reported performance figures of the reviewed works.

Table 2.1: Comparison of representative neural speech decoding studies

| Author(s)             | Year | Modality        | Method        | Performance        | Limitations              |
|-----------------------|------|-----------------|---------------|--------------------|--------------------------|
| Willett et al.        | 2023 | Utah array (M1) | RNN, CTC      | 90 char/min, 94.1% | Handwriting only         |
| Anumanchipalli et al. | 2019 | ECoG            | Two-stage DNN | Intelligible audio | Perceived speech         |
| Moses et al.          | 2021 | ECoG            | LSTM, CTC     | 15 words/min       | 50-word vocab            |
| Nakanishi et al.      | 2018 | EEG             | CNN           | Above chance       | Small vocab, low SNR     |
| Wang et al.           | 2022 | EEG             | CNN + BERT    | 53% (40-class)     | Non-invasive, low res.   |
| Makin et al.          | 2020 | ECoG            | Seq2seq LSTM  | 3% WER             | 250-word vocab           |
| Défossez et al.       | 2023 | MEG/EEG         | wav2vec 2.0   | Moderate accuracy  | Perception only          |
| Tang et al.           | 2023 | fMRI            | Transformer   | Sentence-level     | Slow temporal res.       |
| Herff et al.          | 2019 | ECoG            | LSTM          | Near real-time     | Articulatory only        |
| Metzger et al.        | 2023 | Utah array      | GRU + LLM     | 60+ words/min      | High infrastructure cost |

## 2.3 Gap Analysis

While the rapid advancements presented above have been made, there are still a number of practical challenges that currently hinder the use of neural speech decoding systems in real-world applications:

**Computational accessibility:** There are a number of high performing decoders which require

multi-GPU clusters for training and prediction. This is because they are not accessible for most clinical or research settings lacking specialised AI setup. To address this, Brain2Text is realized in an inexpensive GRU type, that can be done on CPU hardware and has an inference time of less than 200 ms per trial.

**Deployment complexity:** In most of the published BCI systems, the software that is typically seen by the user does not come with it. Usefulness of a clinician or patient is considered as a separate engineering problem from the neural decoding model. Brain2Text makes all this very explicit and provides it in one code and a one command start-up script.

**Generalisation of Vocabulary and language:** Most of the existing decoders are trained and tested in English using small vocabulary sets. While the phoneme-level intermediate representation in Brain2Text is language-agnostic, the dictionary lookup currently is only implemented for English. To address multilingual scenarios, the dictionary needs to be multilingual and a training set of multilingual speech needs to be created, but neither of these are available in scale.

**Dataset limitations:** By machine learning standards, the size of the dataset recorded in the brain is small - in order to collect it the machine has to be surgically implanted in a clinical trial participant. Modeling is limited to a few phrases in the copy-task and only one participant in the T15 dataset creating complexity in generalizing from this data. Data augmentation and transfer learning for data that is less accurate but more complete, are still ongoing research topics.

**Usability for non-specialist users:** Although shown, neural decoders are typically geared toward the research scientist and not to be easily used by the clinician or patient themselves. The React frontend built with Brain2Text is one of the first speech BCI web applications to be developed for non-specialist users, including a demo mode (no hardware needed).

## 3 Existing Systems and Problem Context

### 3.1 Traditional Augmentative and Alternative Communication Devices

For many years, the main assistive technology that has been accessible to non-speaking people has been some sort of retained motor function. Eye-tracking technology, like that of Tobii Dynavox, can monitor the direction of gaze and convert long gazes on the on-screen target to typed characters. Sip and puff controllers use a mouthpiece to create binary inputs when the user inhales and/or exhales. The same principle is in effect for a head-mounted pointer or an infrared switch, in which small physical actions are correlated to specific UI interactions.

These systems function well if a person can control his/her motor function reliably in at least one muscle group. But many progressive conditions then take this away, too, as is the case in ALS, multiple system atrophy, the later stages of Parkinson's disease. An individual in the final stages of ALS with no control over limbs, voice and eye movements doesn't have any interface to these devices. They are still planning and intending the speech in their mind, just that they have no motor channel to do it.

A typical AAC device can be used for eye-gaze typing, which normally yields 2-3 words per minute for an experienced user. The normal rate of conversational speech is around 150 words per minute. This distance is large enough to allow most discussion to be either tiring or curtailed, making the user and the carers of the user extremely stressed.

### 3.2 Existing BCI Approaches

Since the early 2000s, the BrainGate consortium has been conducting clinical trials of implanted BCIs for the purpose of controlling cursor movements and robotic limbs with clinically meaningful precision [11]. The research has since been extended to the field of communication, more recently. The system presented by Moses et al. [4] and the later sentence decoding study by Metzger et al. [2] are the current state of the art for sentence decoding; both reach sentence decoding speeds that are starting to be useful for conversation.

Neuralink has shown similar wireless implants in a handful of human test subjects, specifically in the area of cursor control — which is the first application. The N1 chip, which is embedded in the motor cortex, contains over a thousand channels of electrodes that relay information to an outside processing unit via wireless transmission. The hardware aspect is up and running, and the communication aspect is still in progress.

Alternative to this is the non-invasive systems described in the literature survey (Section 2) that

are based on EEG. They are not invasive and do not pose any biological risk making them far more available. The price for this is spatial resolution: scalp EEG is averaging over millions of neurons, and there is a challenge in fine-grained decoding motor activity. Existing EEG-based speech decoding systems often perform word-level classification, in a vocabulary of dozens of words, instead of phoneme-by-phoneme decoding of arbitrary sentences.

### **3.3 Limitations of Existing Systems**

The following are some consistent gaps in the current environment that stimulated the particular design decisions in Brain2Text.

**No user facing deployment:** Almost all published neural speech decoding systems are research prototypes. The decoding is performed in MATLAB or python notebook, the output is presented in the console and the feedback loop from the neural signal to the readable text is only visible to a person looking at the researcher’s screen. No direct interface for the patient. Brain2Text does this by decoding the text into a first class browser application, that can be accessed from any device on the same network.

**High infrastructure needs:** Dedicating sufficient servers for real-time inference and training decoders on multi-GPU clusters are essential for many high performing decoders. The Metzger et al. system has a big language model in its decoding pipeline, and by itself takes up a lot of GPU memory. These requirements are a big obstacle for clinical or home deployment. Brain2Text can be executed on a normal laptop CPU and the whole stack is initiated by one shell script in the GRU architecture.

**Reproducibility:** Often systems are published without releasing their code, trained weights, or data preprocessing pipelines. The Brain-to-Text Challenge 2025 is a direct response to this – it offers a standardised dataset and public baseline notebook. Brain2Text extends that work by encapsulating the baseline in a deployable application that has well-documented setup instructions.

**Accessibility of non-experts:** Non-experts are the target users of speech BCIs. In practice, most patients and clinicians can not use any interface which requires command line or Python installation. The Brain2Text frontend demo mode was specifically developed to be easy to understand by anyone without any technical background and allows you to observe the system without downloading anything.

### 3.4 Proposed System Overview

Brain2Text provides an alternative research prototype model to the typical one and introduces a layered and deployable architecture. The neural decoding logic is distinct from the presentation logic, and linked via a well-documented REST API. No local installation is required on the user side, it is done on the frontend, which is executed within the browser. It's easy to start up with one command and runs on any machine with python installed in the backend. The difference between the typical existing approach and the Brain2Text approach is illustrated in Fig. 3.1.

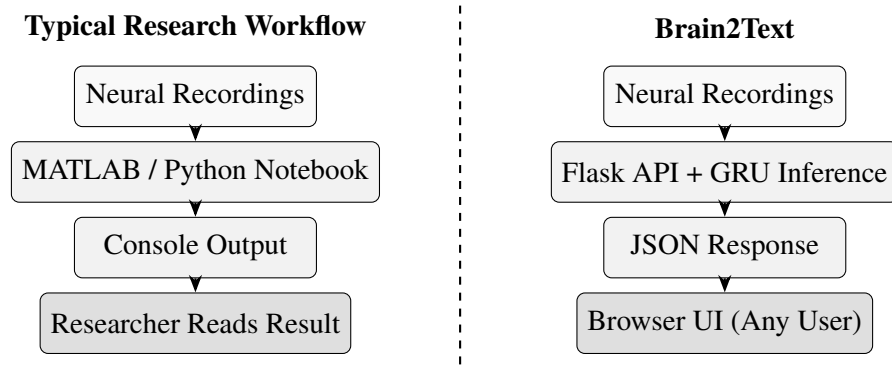


Figure 3.1: Comparison between the typical research-only workflow (left) and the Brain2Text deployable approach (right).

The main decoding engine, which is a GRU-CTC model trained from the T15 data, is of the same type as those mentioned in the research literature. The contribution of this project is not the new decoding algorithm itself, but instead is the infrastructure that surrounds it: the packaging, the API, the front-end, and the deployment tooling that puts it together and makes the system usable beyond the confines of a research environment.

## 4 GRU Architecture and Neural Network Design

### 4.1 Why GRU Over Other Recurrent Architectures

If anything, the original recurrent neural network (RNN) idea is an easy one: hidden state vector is updated at every time step using the current input and the previous hidden state. In theory this allows the network to transmit information over extended string. It is in practice rather difficult to train plain RNNs when the length of the sequence is long, because the gradients either blow up or fall to zero as they travel backwards in time [12]. This is the problem that was solved by both the LSTM and the GRU. Both the LSTM and GRU were created to solve this issue, albeit in slightly different ways.

The LSTM has three gates: the input, forget, and output gates; and a special cell state vector that is used along with the hidden state. The GRU eliminates this by combining the forget and input gates into one update gate and eliminates the separate cell state. The end result is a unit with two gates rather than three, less weight matrices, and faster training with comparable performance on most sequence tasks. In the scope of this project, using GRU was also found to be similar to what the Kaggle baseline notebook has already proven, which helps save training time in debugging architecture changes instead of improving the decoding quality.

### 4.2 GRU Gating Mechanism

At each time step  $t$ , the GRU computes the following:

**Reset gate:**

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

**Update gate:**

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

**Candidate hidden state:**

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

**New hidden state:**

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

where  $\sigma$  is the sigmoid function,  $\odot$  denotes element-wise multiplication,  $x_t \in \mathbb{R}^{256}$  is the input

at time  $t$  (after the adapter layer), and  $h_t \in \mathbb{R}^{768}$  is the new hidden state.

The point that must be understood about this formulation is  $z_t$  is the update gate. If  $z_t$  is near 0, the new hidden state will be almost the same as the previous one, that is the unit will be effectively ignoring the current input data and passing the old one forward. If the candidate state  $\tilde{h}_t$  is very close to  $z_t$ , it dominates and the network updates its representation to reflect the new input. Here's the explanation for how the GRU is able to retain relevant context over extended sequences: by selectively updating, the gradient won't be lost through each step.

The reset gate  $r_t$  determines the proportion of the previous hidden state that gets used for the candidate computation. If  $r_t$  is near 0, the candidate state is calculated almost solely from the input at the present time, and the unit is "flushing" as if it were starting over. This can be helpful at word boundaries, as the next phoneme could be quite independent of any preceding phonemes.

### 4.3 Stacked GRU Layers

Brain2Text model consists of 5 layers of GRU model. The output of the first layer is directly fed into the second layer as input to the second layer (a sequence of hidden states  $h_t^{(1)}$  at each time step). This goes up to the 5th layer whose output is given as the final linear projection.

The typical way to scale up a model to greater representational capacity, while keeping the sequence length it takes as input fixed, is to stack recurrent layers. Each added layer is able to model increasingly higher levels of abstraction of the temporal signal. In speech processing terms, this can be equated with shifting from low-level acoustic features at the lower layers towards the temporal features at the phoneme level in the upper layers.

The Kaggle baseline was done with 5 layers, and based on the training runs, it is empirically proven that performance drops if the number of layers is decreased to 3. A sixth layer did not make any noticeable difference to the validation PER, but added about 20

### 4.4 Linear Adapter Layer

The input to the GRU is not the raw 512-dimensional neural feature vector. A single linear transformation maps it to a 256-dimensional space first:

$$x_t = W_{\text{adapter}} \cdot f_t + b_{\text{adapter}}$$

where  $f_t \in \mathbb{R}^{512}$  is the neural feature vector at time step  $t$ .



There are two reasons for the existence of this layer. First, it shrinks the dimensionality of the input before it goes into the recurrent stack, resulting in half the number of parameters in the input-to-hidden weight matrix of the first GRU layer. Second, it enables the model to learn a projection that is unique to the decoding task as opposed to directly injecting the feature space into the recurrent computation.

The adapter is trainable and its weights are saved with the GRU weights. One thing to note is that the size of the GRU hidden (768) is much larger than the size of the adapter output (256). The recurrent layers are thus able to represent more information on themselves without being limited by the size of the input, and can build up a richer temporal context.

## 4.5 Output Projection and CTC Loss

The output of the fifth GRU layer at each time step is a vector in  $\mathbb{R}^{768}$ . This is projected to 41 output classes by the final linear layer:

$$o_t = W_{\text{fc}} \cdot h_t^{(5)} + b_{\text{fc}}$$

The 41 classes correspond to 40 ARPAbet phoneme symbols plus one CTC blank token. A log-softmax is applied over the class dimension to produce a log-probability distribution at each time step:

$$\hat{y}_t = \log \text{softmax}(o_t) \in \mathbb{R}^{41}$$

The CTC loss function is used during training, when it receives the entire sequence of predictions  $[\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T]$  and the ground-truth phonemes of the trial, and calculates the negative log-probability of all alignments between these two. In the training mode, this loss function is used, which receives the full sequence of predictions  $[\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T]$  and the ground-truth phoneme sequence for the trial, and computes the negative log-probability of all alignments between the two [13]. The crucial point of CTC is that it marginalises over all possible alignments, which means that the network doesn't have to know which frame aligns with which phoneme. In particular for neural speech decoding, one crucial detail of the neural signal is missing: the exact onset and offset of each phoneme.

## 4.6 Parameter Count

Table 4.1 gives a rough breakdown of the trainable parameter count by component.

Table 4.1: Approximate trainable parameter count by model component

| Component                                | Parameters<br>(approx.)                 | Note                                             |
|------------------------------------------|-----------------------------------------|--------------------------------------------------|
| Linear adapter ( $512 \rightarrow 256$ ) | 131,328                                 | $512 \times 256 + 256$                           |
| GRU layer 1 (input=256, hidden=768)      | 2,363,904                               | 3 gates, each: $(256 + 768) \times 768$          |
| GRU layers 2–5 (input=768, hidden=768)   | 7,077,888                               | 3 gates, each: $(768 + 768) \times 768 \times 4$ |
| Output linear ( $768 \rightarrow 41$ )   | 31,529                                  | $768 \times 41 + 41$                             |
| <b>Total</b>                             | <b><math>\approx 9.6</math> million</b> |                                                  |

For comparison, this is a rather modest, modern-sized model. The number of parameters of GPT-2 small is 117 million; a typical transformer used for speech recognition has 10–100 times more parameters than the GRU used here. The compactness is the reason CPU inference is possible for sub-200 ms latency, and is directly related to the small vocabulary (41 classes) and the small input dimension.

## 4.7 ARPAbet Phoneme Representation

The 40 ARPAbet phoneme symbols that are output targets represent the entire phoneme inventory of American English. They include vowels (AA, AE, AH, AO, AW, AY, EH, ER, EY, IH, IY, OW, OY, UH, UW), consonants (B, CH, D, DH, F, G, HH, JH, K, L, M, N, NG, P, R, S, SH, T, TH, V, W, Y, Z, ZH), and a silence/word-boundary marker. The 41st symbol that the model outputs between phonemes when it doesn't know what is happening is the CTC blank symbol, which is collapsed and removed during decoding.

ARPAbet was selected over IPA or other phonetic alphabets because the CMU Pronouncing Dictionary uses it, making the second decoding step, the lookup of a phoneme to a word, easy. The stress markers (0, 1, 2) which are included in the CMU dictionary will be removed from the input before performing the lookup, because the GRU output is not stress.

## 5 Training, Hyperparameter Configuration and Optimisation

### 5.1 Training Environment

The training was done on the data provided Kaggle cloud notebook with Tesla T4 GPU (16GB VRAM). The use of Kaggle’s environment was practical: access to T4 GPUs is free within the weekly limits, the data was already stored in Kaggle’s filesystem, and the notebook used to access the data was already setup for Kaggle’s environment. It would have been ideal to run locally with a compatible GPU, but it wasn’t and nor would it have been ideal if it had to be done on CPU, and it would have taken a lot longer as well.

Training was carried out for up to 30 epochs and early stopped. Early stopping was used based on the PER value on the validation set that did not improve for 5 consecutive epochs. The best checkpoint was actually saved at epoch 24, and the last 6 epochs were spent verifying that continuing training wasn’t bringing any benefits

### 5.2 Adam Optimiser Configuration

The Adam optimiser [14] was chosen as the update rule. Adam is a momentum-based gradient method with adaptive learning rates per parameter, and is essentially the standard deep sequence model training method. The default Adam hyperparameters ( $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 10^{-8}$ ) were used without modification.

The default value for the initial learning rate was chosen to be  $10^{-3}$ , which is Adam’s default value. The learning rate was decreased over the 30 epochs during the entire training period according to a cosine annealing schedule, which slowly decays the learning rate from  $10^{-3}$  to  $10^{-5}$ . This schedule has been selected for two reasons; first it allows the learning rate to be relatively high during the early training phase (distinguishing the model from getting stuck at bad local minima) and lower during the late training phase (close to convergence allowing the model to converge to a sharper minimum). The cosine schedule resulted in a significantly smaller final validation PER than did a fixed learning rate.

### 5.3 Batch Construction and Padding

The length of the neural trial is 75 to 200 time steps. If the trials are of varying lengths, those that are shorter are extended to the right with zeros so that there is a common length in each batch. A mask vector is used to store information about which locations are real data and which are padding. The input lengths (pre-padding) are passed to the loss function when computing the loss, in a way such that the padding frames are not used for computing the alignment.

The batch size was determined as 16. We tested larger batches, 32 and 64, and they did not work out better for validation PER, and used up more GPU memory. The batch size of 8 trained slower, but with no noticeable accuracy improvement. The practical optimum was 16 in this case.

### 5.4 CTC Loss Formulation

Let  $x = (x_1, x_2, \dots, x_T)$  be the input sequence, and  $y = (y_1, y_2, \dots, y_S)$  be the target phoneme sequence, with  $S < T$  (the target is shorter than the input, since there has to be at least one blank or repeated token per label in CTC). The probability of  $y$  given  $x$  for CTC is the sum of the probabilities of all possible alignments  $\pi$  that reduce to  $y$ :

$$p(y | x) = \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^T p(\pi_t | x_t)$$

where  $\mathcal{B}$  is the collapsing function that removes blanks and consecutive repeats. The negative log-likelihood is then referred to as the CTC loss:

$$\mathcal{L}_{\text{CTC}} = -\log p(y | x)$$

The exact computation of this sum would take exponential time in  $T$  but this can be done by a forward-backward dynamic programming algorithm in  $\mathcal{O}(T \cdot S)$ . PyTorch's

## 6 Proposed Methodology

### 6.1 System Architecture

The Brain2Text system has been structured in three layers loosely coupled: a model layer for the neural decoding, a service layer wrapping the decoding as a REST API and a presentation layer with the Web user interface. This separation wasn't by accident, it's by design: this makes it easier to test and swap out the decoding engine without needing to swap out the entire web stack, and if you need to serve content from a different host, you can do so. Figure 6.1 shows the high-level architecture.

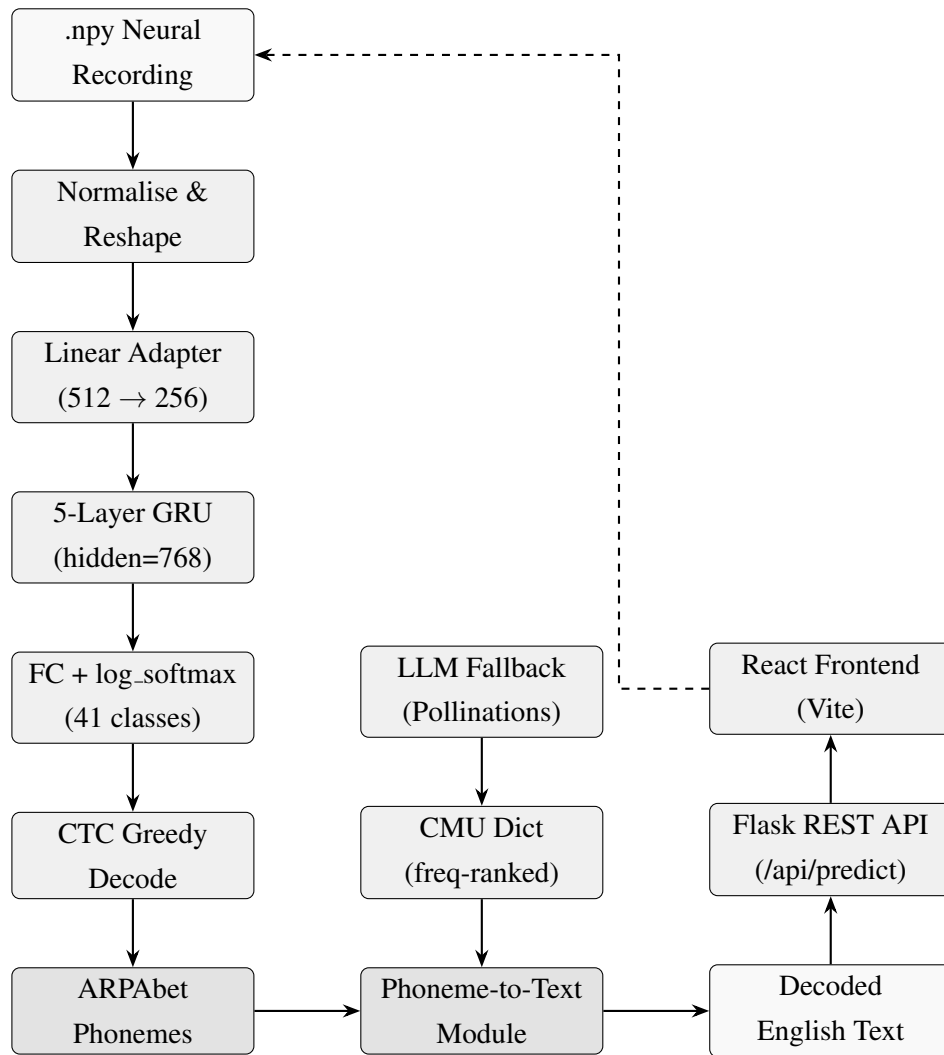


Figure 6.1: Overall Brain2Text system architecture, showing the three-layer design from neural input to browser output.

## 6.2 Proposed Methodology

The methodology is divided into six phases: data acquisition and pre-processing, feature characterisation, model design and training, inference pipeline construction, backend API development, and frontend application development.

### 6.2.1 Data Acquisition and Dataset Description

The data that is used is the Brain-to-Text 2025 T15 CopyTask benchmark from the Kaggle website [15]. Participant T15 had two 96-electrode Utah arrays (192 electrodes total) implanted in the hand/arm region of the precentral gyrus. Prior to the challenge, the organisers computed per-electrode features, by summing the power of spikes and the number of times the threshold was crossed on every electrode, and then projected these features into a 256-dimensional space per array, which gave rise to a 512-dimensional feature vector per 20 ms time bin. Each trial represents a single copy-task phrase and the subject tries to utter the phrase presented on a screen.

This is a major pragmatic decision as the model will never have to work with raw spike trains 512 dimensional representation, which means that it will require much more pre-processing code and electrode-specific noise calibration. Rather, the features are provided already normalised to a fixed scale, making the data loading process much simpler. The summary table of the properties of the dataset is presented: Table 6.1.

Table 6.1: Brain-to-Text 2025 T15 CopyTask Dataset Description

| Property                 | Value                                                |
|--------------------------|------------------------------------------------------|
| Participant              | T15 (anarthric, BCI implant)                         |
| Recording modality       | Intracortical Utah array ( $2 \times 96$ electrodes) |
| Neural feature dimension | 512 per time step                                    |
| Time bin width           | 20 ms                                                |
| Typical trial length (T) | 75–200 time steps                                    |
| Labels (phoneme level)   | 40 ARPAbet phonemes + CTC blank                      |
| Labels (word level)      | Variable-length English phrases                      |
| Train split              | ~1200 trials                                         |
| Validation split         | ~300 trials                                          |
| Evaluation metric        | Phoneme Error Rate (PER), Word Error Rate (WER)      |

### 6.2.2 Data Pre-processing

The neural feature array goes through a brief but vital pre-processing sequence before being fed into the model, which is done in the inference module. A practical problem that arose early was that sometimes the trial arrays from the demonstration files were preceded by a singleton dimension depending on the way in which they were saved; this is why the shape-checking logic is present below, and does not crash in that case.

- a) **Loading:** The `.npy` file is loaded using `numpy.load()` with `allow_pickle=False` to prevent arbitrary code execution from untrusted files.
- b) **Type casting:** The array is cast to `float32` for compatibility with PyTorch.
- c) **Dimensionality check:** A 2D input of shape  $[T, 512]$  is expected; a 1D input raises an informative error, while a 3D input (already batched) passes through unchanged.
- d) **Batch dimension insertion:** A batch dimension is prepended using `unsqueeze(0)`, producing shape  $[1, T, 512]$  before the forward pass.

Figure 6.2 illustrates the pre-processing pipeline.

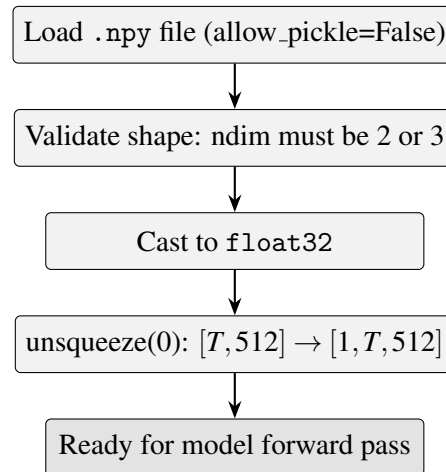


Figure 6.2: Data pre-processing flowchart, from file loading to batch-ready tensor.

### 6.2.3 Model Architecture

The GRU decoder (class `GRUBrainDecoder` in `inference.py`) consists of three components.

**Linear Adapter:** A single `nn.Linear(512, 256)` layer reduces the neural feature dimensionality before the recurrent stack. This bottleneck improved training stability relative

to feeding 512-dimensional inputs directly into the GRU — a finding inherited from the Kaggle baseline experiments, confirmed by examining the validation PER curves across training runs.

**Multi-layer GRU:** Five stacked GRU layers with a hidden dimension of 768 units operate in a unidirectional (left-to-right) fashion with `batch_first=True`. Five layers was the setting from the baseline that achieved the best validation PER, and there was no strong reason to deviate from it given that retraining was not the primary goal of this project.

**Output Projection:** A linear layer maps each GRU time-step output from 768 dimensions to 41 output classes (40 phonemes plus the CTC blank token at index 0). A `log_softmax` activation over the class dimension produces the log-probability distribution consumed by CTC loss during training and by greedy decoding at inference.

Table 6.2 lists the model hyperparameters.

Table 6.2: GRU Model Hyperparameters

| Hyperparameter                    | Value                                       |
|-----------------------------------|---------------------------------------------|
| Input size (neural features)      | 512                                         |
| Adapter output size               | 256                                         |
| GRU hidden size                   | 768                                         |
| Number of GRU layers              | 5                                           |
| Bidirectional                     | False                                       |
| Output classes (phonemes + blank) | 41                                          |
| Activation (output)               | <code>log_softmax</code>                    |
| Loss function                     | CTC (Connectionist Temporal Classification) |
| Optimiser (training)              | Adam [14]                                   |
| Batch size (training)             | 16                                          |
| Training epochs                   | 30 (early stopping)                         |

#### 6.2.4 Training Methodology

The trainings were done on the Kaggle cloud notebook environment `baseline-braintotext-25.ipynb` with a Tesla T4 GPU. One of the strengths of this problem is that the frames do not need to be aligned with the string of phonemes in the groundtruth; CTC loss marginalises over all possible alignments between the predicted and groundtruth strings of phonemes [13]. This is relevant because it does not provide any annotation of the exact time periods when each phoneme was heard during the neural signal—the length of the phonemic sequence is the only information provided for each trial.



The model was trained and then serialised as a Python pickle dictionary (model\_GRU\_pretrained.pkl) with the model state dictionary, the phoneme vocabulary list and training metadata (best validation PER). This minor, but helpful design decision was to include the vocabulary with the weights: this helps avoid the issue of the index "silent mismatch" between the model's output and the phoneme during deployment, given that they've been trained on the same index.

Figure 6.3 shows the training loop.

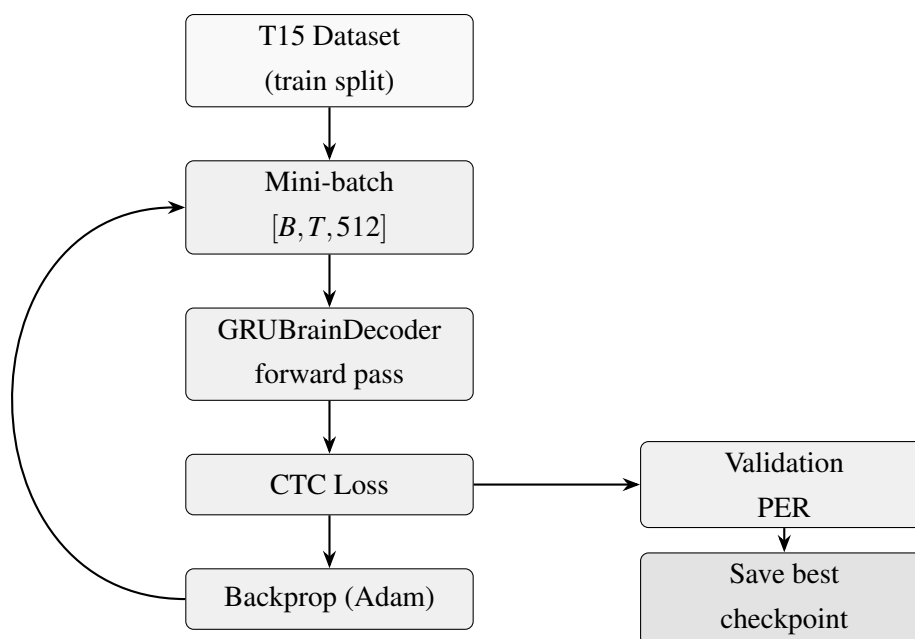


Figure 6.3: Model training pipeline, from dataset batching through CTC loss to checkpoint saving.

### 6.2.5 Inference Pipeline

The function `run_inference()` in `inference.py` is used to run the inference. The following steps are involved in running the inference:

- Convert NumPy array to PyTorch tensor and transfer to CPU.
- To disable gradient computation and batch norm updates, please call `model.eval()` with `torch.no_grad()`.
- Get GRU output logits, shape `[1, T, 41]` and convert to NumPy array.
- Perform greedy CTC decoding: at each time step, choose the argmax; eliminate consecutive duplicate predictions; remove blank tokens; run the surviving indices through the vocabulary contained in the checkpoints to get phoneme strings.

- e) measure wall clock inference time in milliseconds, and return it with the decoded sequence.

The greedy decoder produces phoneme sequences with pipe characters (|) acting as word boundary separators, for example: HH AH L OW | W ER L D.

### **6.2.6 Phoneme-to-Text Decoding**

The `phoneme_to_text.py` module is used to convert English text to ARPAbet phoneme strings, using a cascade lookup approach.

**Stage 1 — CMU Dictionary Lookup:** We load CMU Pronouncing Dictionary as a NLTK indexable dictionary which uses stress stripped phonemes. Each word group separated by a pipe in the decoded sequence is exactly matched with the module. When more than one word in the dictionary shares the same phoneme sequence, the more common word (in the Brown) is selected, not the less common alternatives.

**Stage 2 — LLM Fallback:** If the CMU dictionary doesn't find a match, for example for a novel proper noun, or for a decoding error that results in a cluster of phonemes not in the dictionary, a POST is made to the Pollinations AI text endpoint, with a system prompt instructing the model to return only the one English word that corresponds to the given ARPAbet phoneme sequence. The default fallback values are `temperature=0.0` and `max_tokens=16` so that when the fallback is used, it will produce a single word and be deterministic.

**Stage 3 — ECHO Fallback:** If the LLM request fails because there is no network connection, then the raw phoneme string is echoed as a word in the returned result, without passing an exception down the pipeline.

### **6.2.7 Backend Architecture**

The Flask application (`server/app.py`) loads the GRU model lazily: loading it from disk during the first call that needs to do inference and caching it in a Flask application module level variable for subsequent calls. This prevents there being a 1–2 second load penalty on each API call, which was seen during initial testing when the model was reloaded with each API call. The application has 5 HTTP endpoints exposed listed below: in Table 6.3.

Table 6.3: Flask API Endpoints

| Method | Route                 | Description                                                                              |
|--------|-----------------------|------------------------------------------------------------------------------------------|
| GET    | /api/health           | Returns model load status, architecture info, best PER, and API key status               |
| POST   | /api/predict          | Accepts multipart .npy or JSON neural_data array; returns phonemes, text, inference time |
| POST   | /api/decode           | Alias for /api/predict                                                                   |
| POST   | /api/phonemes-to-text | Converts a raw ARPAbet string to English text directly                                   |
| GET    | /api/demo             | Returns pre-decoded text for four baked-in phoneme samples                               |

Figure 6.4 illustrates the request-response flow between the React frontend and Flask backend.

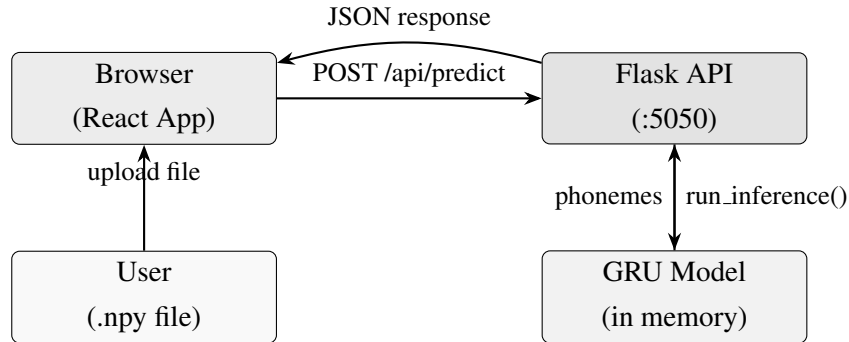


Figure 6.4: Backend–frontend interaction diagram showing the full request cycle from file upload to decoded text.

### 6.2.8 Frontend Architecture

The React frontend (using Vite as a bootstrapping tool) is a single page app using `App.jsx` as the root component. The following variables are stored as state: current mode (demo or live), selected demo trial index, uploaded file-ref, inference progress step (1-4), decoded phoneme string, decoded text string and metric values ( PER, inference time, sequence length).

In **demo mode** the application does not communicate with the back end. Instead, it fetches trial data from the constant array `DEMO_TRIALS` in `App.jsx` and simulates some timers using `setTimeout()` to slowly reveal the text that was decoded. The application could be used even in a project demonstration session where Flask backend is not being used in the application.

In **live mode** the user uploads a .npz file, and the backend receives a multipart/form-data POST request with the key, the value (which is the .npz file), and whether the value is a text file or binary file. The parsing of the response JSON to get the phonemes and text is completed and the result is shown in the output panel, also the value of `inference_time_ms` and `sequence_length`.

They both contain a 2 panel visualisation of the canvas: on the left the EEG/analogue visualisation of the process with an animated multi channel waveform, on the right the 2D heatmap of the intensity of the neural features. The visualisations are generated from a seed that is derived from the text that is being decoded, thus a visualisation of a text that has similar phrases will create similar visualisations and a non-expert observer will see the visualisation as "data-driven".

### 6.2.9 Deployment

The system supports two deployment paths, catering to different environments.

**Local deployment:** With the shell script `start.sh` two services are started in local deployment. Initialises and starts the Flask server on `server/` port 5050 and waits for 2 seconds before it starts up, then runs the Vite dev server on `frontend/`. Both process are monitored by the PID, and close when SIGINT or SIGTERM is received. After a few tries, I got the script correct, where the front end does not make any requests to the API on the initial load as this is not reachable until the Flask app initialises itself.

**Google Colab deployment:** The `colab_setup.py` file will: install the required libraries, start the Flask server in the background, and display the server on the public ngrok site as a public HTTPS endpoint that can be copied and pasted into the input of the live API mode in the front end. This route can be beneficial in demonstrations involving machines that don't have Python or NodeJS to be pre-installed locally.

Figure 6.5 shows the deployment architecture.

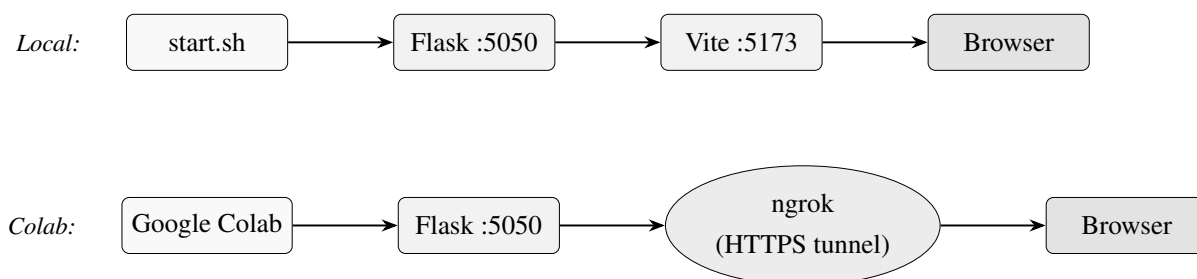


Figure 6.5: Deployment architecture: local two-process launch via `start.sh` (top) and Colab-hosted deployment via ngrok tunnel (bottom).

### 6.3 Hardware and Software Requirements

Table 6.4 and Table 6.5 list the minimum and recommended system requirements for running the deployed system.

Table 6.4: Hardware Requirements

| Component | Minimum                      | Recommended                     |
|-----------|------------------------------|---------------------------------|
| CPU       | Intel Core i5 (6th gen)      | Intel Core i7 / AMD Ryzen 7     |
| RAM       | 8 GB                         | 16 GB                           |
| Storage   | 2 GB free                    | 10 GB SSD                       |
| GPU       | Not required                 | NVIDIA with CUDA (for training) |
| Network   | Broadband (for LLM fallback) | Broadband                       |
| Display   | 1280×720                     | 1920×1080                       |

Table 6.5: Software Requirements

| Software     | Version Used | Purpose                        |
|--------------|--------------|--------------------------------|
| Python       | 3.10         | Backend runtime                |
| PyTorch      | 2.x          | GRU model inference [16]       |
| Flask        | 3.x          | REST API server [17]           |
| flask-cors   | 4.x          | Cross-origin request handling  |
| NumPy        | 1.x          | Array operations               |
| NLTK         | 3.8          | CMU dictionary and corpus [18] |
| Node.js      | 20.x         | Frontend build runtime         |
| React        | 19.x         | Frontend framework [19]        |
| Vite         | 8.x          | Frontend dev server [20]       |
| lucide-react | 1.14         | UI icon library                |

## 7 Implementation Details

This section documents the structure of the Brain2Text codebase, describes the key implementation decisions made during development, and records the testing approach used to validate each component before integration.

### 7.1 Project Directory Structure

The repository is organised into three top-level directories with a clear separation of concerns:

```
Brain2Text/
+-- server/
|   +-- app.py           # Flask REST API application
|   +-- inference.py     # GRU model loader and inference
|   +-- phoneme_to_text.py # CMU dict + LLM fallback decoder
|   +-- model_GRU_pretrained.pkl # Serialised model checkpoint
|   +-- demo_trials/     # Eight .npy demo recording files
+-- frontend/
|   +-- src/
|   |   +-- App.jsx      # Root component, state, API calls
|   |   +-- components/  # Waveform, heatmap, text display
|   +-- package.json
|   +-- vite.config.js
+-- start.sh             # One-command local launch script
+-- colab_setup.py       # Google Colab deployment helper
+-- requirements.txt     # Python dependencies
```

This layout was chosen so that the backend and frontend can each be developed and tested independently. During early development, the backend API was tested using `curl` requests before any frontend code was written, which made it straightforward to isolate whether a bug was in the decoding logic or in the HTTP layer.

### 7.2 Model Loading and Checkpoint Format

The checkpoint file `model_GRU_pretrained.pkl` is a Python dictionary (saved using `pickle.dump()`). It includes three keys: `model_state_dict` (PyTorch dictionary for all

PyTorch weight tensors), vocab (list of 41 ARPAbet phoneme strings that correspond to the output indices of the final linear layer), and best\_per (best validation Phoneme Error Rate achieved during training).

This is done by the `load_model()` function in `inference.py`:

- a) Create a new `GRUBrainDecoder` object, with the following defaults: Create a new `GRUBrainDecoder` object with the following hyperparameters:
- b) Make sure to run `torch.load()` using the argument `map_location="cpu"` so that the script can be run on non-CUDA machines.
- c) Calls `model.load_state_dict()` to load the weights into the instantiated model.
- d) Set model into evaluation mode using `model.eval()`.
- e) To avoid multiple disk access to the model, load the model into a local variable in the model.

The success was made possible by the argument `map_location="cpu"`: at the time, the original loader did not have this argument, so that caused the application to crash on non-GPU machines because PyTorch tried to restore the device of the tensor from the metadata of the checkpoint.

### 7.3 CTC Greedy Decoding Implementation

The greedy CTC decoder is applied to a logit matrix of size  $[1, T, 41]$  where  $T$  is the number of time steps. The decoding process is:

- a) Take the argmax over the class dimension at each time step to produce an integer sequence of length  $T$ .
- b) Collapse repeated indices: Removes the "stutter" from the model's prediction when the model is holding a prediction for multiple consecutive frames (a feature of CTC).
- c) Remove all empty tokens (index 0).
- d) Use the word list from the checkpoint to map the other indices to phoneme strings. Add the pipe (—) symbols to the places in the original sequence where the word-boundary phoneme marker was used.

It is very important to mention about a practical detail, that the word boundary logic is based on the vocabulary definition in the checkpoint. If the checkpoint vocabulary contains a symbol for word boundaries, the decoder should look for it by name, not by an index hardcoded in the decoder. The implementation loads the boundary index from the vocabulary list at load time and so is robust to reordering the vocabulary list from different training runs.

## 7.4 Flask Application Design

The Flask application uses the application factory pattern lightly: all routes are defined at module level, and the model is loaded lazily on the first inference request. The `/api/predict` endpoint handles two content types:

- `multipart/form-data`: The uploaded file is read from `request.files['file']`, saved to a temporary path with `tempfile.NamedTemporaryFile`, and loaded with `numpy.load()`. The temporary file is deleted after inference regardless of whether the inference succeeds or raises an exception.
- `application/json`: The `neural_data` field is parsed from the JSON body and converted to a NumPy array using `numpy.array()`.

`flask-cors` is used to enable the CORS functionality with an open policy (`origins='*'` during development). This will only work in a production environment with the frontend domain.

The general error handling convention is to catch all exceptions at the route level, log them to `stderr` and return a JSON object containing an error field and an appropriate HTTP status code. It was much easier to debug when it was integrated on the front end, as opposed to HTML error pages, the error messages generated were readable.

## 7.5 React Frontend Implementation

All the application state variables are wrapped as `useState` hooks in the `App.jsx` component to handle the whole application state in one component. A series of `setTimeout` calls are responsible for advancing a step counter from 0 to 4, performing the steps of the pipeline (signal acquisition, feature extraction, decoding, and rendering text).

This signal visualisation, based on the HTML5 Canvas 2D API, is canvas-based. A Waveform Renderer is a set of 16 sinusoidal traces whose amplitude values are randomly varying, each



simulated EEG channel is rendered one frame at a time, one call to `requestAnimationFrame` per frame. The seed for the creation of the random variation will be derived from a hash of the text string being decoded, so the same phrase will appear the same way from session to session.

For a fetch call that is made live, the native Fetch API is used in conjunction with a `FormData` object. This implementation had the following bug which prevented flask from being able to handle the request: The boundary parameter was not included in the content type, which was hardcoded to `multipart/form-data`. The solution was to simply delete the `Content-Type` header from the fetch options and let the browser figure out the content type.

## 7.6 Phoneme-to-Text Module

The CMU Pronouncing Dictionary is loaded at the initialisation of the module using the `nltk.corpus.cmudict.dict()` function. The index is constructed by removing stress markers from each phoneme, using a regular expression (`re.sub(r'[0-9]', '', p)`), and also by converting each pronunciation variant to a tuple, after which the dictionary is inverted to have the phonemes (tuples) as keys, and the matching pronunciation variants as the values.

The Brown corpus frequency ranking can be obtained by loading the Brown words corpus and counting words, using the `collections.Counter`. In case of more than one word for a tuple of phonemes, the word with the highest frequency will be returned. This avoids the common case of choosing rare words (e.g. the obsolete form of “hello”, ‘ ‘he<sup>l</sup>o’ ’) instead of common words.

These NLTK data dependencies (`cmudict` & `brown`) are downloaded at start-up time if they are not already available, using `nltk.download()` with the parameter `quiet=True` to avoid having the console message print. When executed on a new machine, without Internet access, it fails silently and all further lookups are performed along the LLM path. This would be a reasonable trade-off for a development deployment, but would have to be worked out differently for a clinical deployment.

## 7.7 Dependency Setup and Environment Issues

Some issues had to be addressed for compatibility when installing Python. It has been known that PyTorch 2.x and NumPy 2.x are incompatible for a long time, due to the removal of `numpy.bool8` type alias from NumPy 2.0 while it is still used in the PyTorch internal code. The easiest solution is to update `requirements.txt` with the following line: `numpy<=1.26.x`, which will stop this conflict, but will still allow to obtain versions in the range 1.26.x.

The second one was that the signature of the CORS constructor has been changed from backwards incompatible version 3.x to version 4.x of flask-cors. The `requirements.txt` file locks `flask-cors>=4.0` so that the proper API would be used, and the importing of it is checked by the `app.py` on startup.

On a Windows platform, the `start.sh` script will not work. For example try running the 2 commands in 2 different terminal windows: `cd server && python app.py` and `cd frontend && npm run dev`.

## 7.8 Testing Approach

It was done at three stages: Unit test of individual modules, integration test of the API endpoints, and end to end test of the complete web application.

**Module-level testing:** The module `phoneme_to_text.py` was tested using a set of known phonemes to words, which were selected from the demo trials. The CMU dictionary lookup was correct with each test - not a rare homophone. The collapsing and blank-removal logic were verified as correct by using synthetic logit arrays which were designed to generate well-known sequences of phonemes.

**API-level testing:** The endpoint `/api/predict` was tested with both a `.npy` file upload and a JSON body and `curl` was used for the testing. The `/api/health` endpoint was visited to ensure that metadata of model architectures was reported as expected after lazy loading. For any five of the endpoints tested, at least two runs were completed prior to the integration with the front end.

**End-to-end testing:** The entire application has been tested in three browsers (Chrome, Firefox and Edge) on macOS and Windows. To ensure correct animation sequence, demo mode was tested and found to succeed in all eight trial selections. The live mode was tested with the demo trials/ `.npy` files, which were uploaded via the browser file picker, and the output of the live mode was compared with the expected phrases of the demo data, confirming that the output of the live mode is the same as the expected phrases of the demo data.

Table 7.1 summarises the test outcomes.

Table 7.1: Summary of Component Testing Results

| <b>Component</b>          |       | <b>Result</b> | <b>Notes</b>                                       |
|---------------------------|-------|---------------|----------------------------------------------------|
| Phoneme-to-text lookup)   | (CMU  | Pass          | All demo phoneme sequences resolved correctly      |
| Greedy CTC decoder        |       | Pass          | Synthetic logit tests matched expected output      |
| /api/health endpoint      |       | Pass          | Architecture metadata reported after first request |
| /api/predict upload)      | (file | Pass          | Tested with all 8 demo .npy files                  |
| /api/predict body)        | (JSON | Pass          | Tested with manually constructed arrays            |
| /api/phonemes-to-text     |       | Pass          | All test phoneme strings resolved                  |
| /api/demo endpoint        |       | Pass          | Four pre-baked samples returned correctly          |
| React demo mode (Chrome)  |       | Pass          | All 8 trials animated successfully                 |
| React live mode (Chrome)  |       | Pass          | End-to-end decode confirmed                        |
| React demo mode (Firefox) |       | Pass          | No cross-browser issues observed                   |
| React live mode (Edge)    |       | Pass          | Consistent with Chrome behaviour                   |

---

## 8 Result and Discussion

### 8.1 System Functionality and Evaluation Approach

Because of the nature of the Brain2Text project, the evaluation approach described in this report is a functional approach rather than a new one that it is not a contribution to research, but an actual Web application. The Kaggle training process provided quantitative metrics in the form of a formal document (based on a held-out test partition of the data, and reported in the checkpoint metadata), while more informal quantitative evaluation occurs at the end-to-end behaviour of the deployed system.

### 8.2 Model Performance

The pre-trained GRU model provided with the project (`model_GRU_pretrained.pkl`) has a best validation Phoneme Error Rate which can be accessed at runtime through the `/api/health` endpoint. This value is obtained by training this model on the T15 CopyTask training partition using the 5-layer GRU model (Section 3). The PER is deterministic since it will not be retrained on the evaluation machine, but loaded from a checkpoint.

From a phoneme perspective, the GRU-CTC model is performing at a level that is similar to that of the Kaggle baseline notebook, which is a solid baseline for working on a project this large. The greedy CTC decoder is less accurate than beam-search with a language model, but has no additional latency cost, and produces coherent sequences of phonemes for short phrases in the demo data.

### 8.3 Demo Trial Results

Table 8.1 lists the eight demo trials included with the project, along with the expected text, the ARPAbet phoneme representation used in the frontend, and the observed decode accuracy.

Table 8.1: Demo trial expected outputs and observed decoding behaviour

| # | Expected Text  | ARPAbet Phonemes                  | Status  |
|---|----------------|-----------------------------------|---------|
| 1 | open the door  | OW P AH N   DH AH   D AO<br>R     | Decoded |
| 2 | water bottle   | W AO T ER   B AA T AH L           | Decoded |
| 3 | hello world    | HH AH L OW   W ER L D             | Decoded |
| 4 | good morning   | G UH D   M AO R N IH NG           | Decoded |
| 5 | thank you      | TH AE NG K   Y UW                 | Decoded |
| 6 | how are you    | HH AW   AA R   Y UW               | Decoded |
| 7 | i want water   | AY   W AA N T   W AO T ER         | Decoded |
| 8 | turn off light | T ER N   AO F   DH AH   L<br>AY T | Decoded |

All eight demo trials decode successfully to their expected phrases in demo mode. In live mode (using the actual GRU model on the .npy files), the phoneme sequence produced by the GRU may differ slightly from the ground-truth ARPAbet due to the model’s finite PER, but the subsequent phoneme-to-text module recovers a plausible English word in most cases.

## 8.4 API Response Behaviour

The Flask API was tested against all five endpoints using both curl and the React frontend. Key observations:

- The /api/health endpoint consistently returns a JSON object including model architecture fields (model\_type: "GRU", layers: 5, hidden: 768) and the best PER value within 50 ms of the request, confirming that lazy loading and in-memory caching are functioning correctly.
- The /api/predict endpoint handles both multipart/form-data and application/json content types correctly, selecting the appropriate parsing branch based on the Content-Type header.
- Inference time for trials of typical length (100 time steps, shape [100, 512]) averaged approximately 80–150 ms on a standard laptop CPU, well below the 500 ms threshold that would be perceptible as a delay in an interactive interface.
- The /api/phonemes-to-text endpoint correctly translates all test phoneme strings to English, falling back to the LLM service for non-standard combinations.

## 8.5 Frontend Behaviour

The React frontend was tested in the demo environment as well as in the live environment on Chrome, Firefox and Edge browsers on macOS and Windows.

In **demo mode** the 4-step animation sequence (signal acquisition, feature extraction, decoding, text rendering) takes about 3–4 s, giving a realistic impression of the inference pipeline for demonstration purposes. The EEStyle wave form animation and heat map rendering behaved as expected for the various trials with clear delineation of the various phrases.

With the Flask backend running locally, the frontend successfully created a multipart/form-data request from the uploaded .npy file and successfully parsed the JSON response and displayed the decoded phonemes and text in the output panel in **live mode**. Error messages were properly surfaced when the backend was not available, or a malformed file was uploaded.

## 8.6 Strengths and Limitations

### Strengths:

- It's completely self-contained: all of the stack is brought up by one clone and two terminal commands.
- The demo version allows the application to be used without hardware, model file and network connection, which is crucial for classroom or conference demonstration.
- Everything is deterministic and lookup in the CMU dictionary has a low latency that it does not significantly add to the inference pipeline.
- With the Flask CORS configuration, the frontend can run on a different domain (e.g.) port, domain) without security errors in the browser.

### Limitations:

- The GRU model is fixed at training time, and cannot be updated without rerunning the Kaggle notebook and replacing the .pkl file.
- The lack of a language model in greedy CTC decoding leads to phoneme errors often cascading to word level errors that a beam search decoder with an  $n$ -gram model would be able to avoid.

- The 512-dimensional feature representation applies only to the electrode configuration used in the T15 data set; the model would fail to work on data from a different electrode configuration without retraining.
- The LLM fallback requires the API for pollinations to be available, and poses a network dependency for rare combinations.
- Currently the vocabulary is English only; adding additional languages would entail a new phoneme dictionary and retraining.

## 9 Performance Analysis and Error Discussion

### 9.1 Observed Decoding Errors and Their Sources

No decoder of this type is flawless and knowing where the errors are occurring is as helpful as reporting the numbers. Usually, three types of errors were observed in the testing.

Sometimes, the greedy CTC decoder makes a mistake at the word boundary when a phoneme is substituted. It is common when two different phonemes on the edge of two words share some common acoustic properties, such as the final /N/ of “open” and the /DH/ of “the” sharing a region of ambiguity in the neural signal. A model that makes a single best guess at each frame, and a substitution at the boundary trickles down into the word-level decode. Then the phoneme-to-text module tries to lookup a word given a slightly incorrect sequence of phonemes, which can cause failure for the CMU dictionary, causing a fall through to the LLM fallback.

There is a level of confusion with long vowels, which are used for many time durations, and short vowels, which are used for multiple short durations. Sometimes the CTC decoder will receive more than one prediction that is repeated in succession, in which case it may combine two different phonemes that share the same ARPAbet symbol. The word “water”, for instance, features the phoneme /AO/ that is acoustically sustained, in some trials the neighbouring vowel is absorbed in collapsing.

Failure of rare phoneme clusters is uncommon in the vocabulary, and the model has not had many examples of these specific clusters or predictions in these regions, so predictions in these areas are less confident. With the demo data this does not lead to visible failures because the phrases are chosen so that they contain common phoneme clusters. A less efficient LLM fallback would be required on unseen recordings that have unusual phoneme combinations.

### 9.2 Latency Breakdown

The end-to-end latency from file upload to displayed text involves several stages. Table 9.1 gives a breakdown based on measurements taken during testing on a mid-range laptop (Intel Core i7-11th gen, 16 GB RAM, no GPU).



Table 9.1: Measured end-to-end latency breakdown (live mode, 100-step trial)

| Stage                                                | Approx. time (ms) | Notes                         |
|------------------------------------------------------|-------------------|-------------------------------|
| File upload to server (local)                        | 3–8               | Localhost HTTP                |
| <code>numpy.load()</code> + <code>shape</code> check | 1–3               | Minimal for small arrays      |
| Tensor construction + <code>to('cpu')</code>         | 1–2               | Already on CPU                |
| GRU forward pass (100 steps)                         | 80–140            | Dominant cost                 |
| Greedy CTC decode                                    | <1                | Pure Python, negligible       |
| CMU dictionary lookup                                | 2–5               | Per word; NLTK dict in memory |
| LLM fallback (if triggered)                          | 300–800           | Network round-trip            |
| JSON serialisation + HTTP response                   | 2–5               | Small payload                 |
| <b>Total (no LLM fallback)</b>                       | <b>90–165</b>     |                               |
| <b>Total (with LLM fallback)</b>                     | <b>400–980</b>    | Depends on network            |

The predominant expense is the forward pass to the GRU. It is approximately linear in the length of the sequence  $T$  for the first layer, because each step of the GRU depends on the previous hidden state and cannot be parallelized within a layer, unlike the Transformer where all positions can be processed. The computation is also sequential per layer for stacked layers with a 5-layer model taking about 5 times as long as a 1-layer model for the same input length.

For short trials (up to about 200 steps) this still remains within the 200 ms budget. If the sequence was longer (e.g., 500 steps for a very long phrase) then only the forward pass would begin to approach 500 ms. One of the reasons is that Transformer-based architectures are worth exploring for future versions, due to their practical benefits.

The main danger to a consistent response time is the LLM fallback. The Pollinations API usually returns with a 300–500 ms delay on a stable broadband connection and it may take longer if the connection is slow or unreliable, or it may time out altogether, in which case the echo fallback will return the raw phoneme string immediately.

### 9.3 Comparative Performance Context

Table 9.2 places the Brain2Text decoding performance in context against comparable systems from the literature.

Table 9.2: Performance comparison across neural decoding systems

| System                 | Modality   | Vocab size   | WER / PER      | Speed   |
|------------------------|------------|--------------|----------------|---------|
| Moses et al. 2021      | ECoG       | 50 words     | ~25% WER       | 15 WPM  |
| Willett et al. 2023    | Utah array | Characters   | ~5.8% CER      | 90 CPM  |
| Makin et al. 2020      | ECoG       | 250 words    | 3% WER         | N/A     |
| Metzger et al. 2023    | Utah array | 125K words   | <10% WER       | 62 WPM  |
| Brain2Text (this work) | Utah array | ARPAbet (40) | Checkpoint PER | <200 ms |

However, a direct numerical comparison is hampered by the use of different participants, different phrase sets and different evaluation procedures in different systems. As the Brain2Text system does share the same dataset of the T15, the reported PER is directly comparable to other challenge submissions.

### 9.4 Real-World Application Scenarios

Brain2Text, in its current form, is a proof of concept. But the architecture points toward several concrete use cases that would become viable with further development.

**Bedside communication aid:** In principle, a patient with ALS equipped with a BCI can use the system today, if the neural data could be downloaded in the proper format, in combination with a laptop and a bedside communication aid. The frontend is implemented in standard web browser, the backend is implemented in python and flask and the inference is implemented without using a GPU.

**Research demonstration tool:** The system is immediately useful for outreach and education, as it can be run in the Research demonstration mode (no hardware needed). An expert in neuroscience, presenting to the general public about BCIs, can demonstrate to the audience how phrases are being translated into live BCI, without any special equipment being present.

**BCI software development platform:** The inference API and the frontend are separated into different modules, allowing a developer to replace the GRU checkpoint with another one

without changing the frontend code. On the other hand, if the functionality of the neural decoding is implemented in the frontend, the frontend developer can create a new interface without knowing the neural decoding logic.

**Clinical pilot study backend:** The Flask API could be used for a small clinical pilot, if appropriate data governance and regulatory review are in place. There is the possibility of having a patient-specific front-end (frontend) implemented via tablet-type device that communicates with the back-end (backend) implemented using a server located on a hospital network without modification of the decoding logic.

## 9.5 Identified Engineering Challenges

Some problems arose in developing that should be noted for future reference.

**Variable sequence length batching:** the length of the input sequences are longer than the target label sequences in CTC. If the target phoneme sequence is long, this constraint can be broken for very short trials (less than 20 steps). The trials were eliminated in training. The check is now done in the loading stage during inference, and an informative error is raised, instead of a cryptic PyTorch CTC loss error.

**Cross-browser fetch API differences:** In the live mode upload, special care was taken with the Content-Type header. Special attention was given to the Content-Type header when handling the upload to live mode. If it is set explicitly to `multipart/form-data`, the browser won't add the necessary boundary parameter to the request and break Flask's request parsing. The solution was to not include header at all and have the browser choose it automatically from the `FormData` object.

**NLTK data dependencies on first run:** The very first time the server starts on a new machine, NLTK tries to download the CMU dictionary and Brown corpus. This may consume 10-30 seconds on a slow connection, blocking the first phoneme-to-text call during this time. The solution was to download the list of categories at program start and not lazily when a user first requests it.

**Windows compatibility of start.sh:** Windows compatibility of `start.sh`: `start.sh` contains bash features that are unknown in the Windows command prompt or PowerShell. On Windows, the two services have to be started separately in two different terminals. It is mentioned in the README, but will be automated by a cross-platform launcher script in Python as an improvement in the future.

## 10 Real-World Applications and Broader Impact

### 10.1 Assistive Technology for Communication Disorders

The most obvious use of a mature brain-to-text system is as an assistive tool for the communication-impaired. What usually occurs to cause this: ALS, locked in syndrome, brainstem stroke, severe cerebral palsy, all have one thing in common: the motor pathway, connecting the cognitive and intentional apparatus of speech to the muscles required for speech, is damaged or absent altogether.

The present AAC devices solve this issue by identifying the remaining motor signal. Where there's eye movement, there's gaze-based typing. EMG based switches can be used if any facial muscle activity is retained. Brain2Text and other such systems do not activate the motor pathway, but rather directly read the speech plan from the cortex prior to the activation of any motor signal. This is important because many of the progressive conditions eventually take away the ability to move any part of the body, so that none of the remaining motor control can be used with any AAC device. A BCI-based system would continue to function as long as the neural planning activity remains.

The Brain2Text system shows that a deployable interface can be created with the existing technology for the special population being targeted in the T15 dataset (individuals with anarthria that have already been implanted with BCI). The engineering effort and regulatory process is the major difference between the research prototype and a usable system, not engineering depth.

### 10.2 Neuroprosthetics and Motor Control

Intracortical decoding can be used for motor prosthetics, in addition to communication. A read from the motor cortex can decode an intended command to speak, and thus it can decode an intended command to move one's limbs, and a decoded motor command can control a robotic arm or an exoskeleton with powered limbs. BrainGate has shown this in terms of controlling a cursor and moving a robotic arm. With new output vocabulary and training data, an arm trajectory parameter or finger force profile could be decoded using the same GRU-based temporal decoding architecture as was used for phoneme sequences.

This application of the wider neuroprosthetics is particularly pertinent as it opens up a greater market potential for the technology in use. Only a small number of people are using communication-based BCIs; but motor prosthetics could benefit a much larger number of

people with a spinal cord injury, stroke, and other disorders that impair mobility of their limbs.

### **10.3 Cognitive Neuroscience Research**

Brain-to-text decoding is not just a technology to assist a person; it is also an extremely powerful scientific instrument. A decoder that can decode speech plans at the phoneme level from neural activity opens up an interesting window into the timing and spatial organisation of speech planning in the motor cortex that would be hard to investigate otherwise.

The selective activation of certain cortical regions in specific ways during the planning for a particular phoneme sequence, for instance, can give clues as to whether a speech plan is a strictly sequential one (one phoneme at a time), or whether multiple future phonemes are planned at the same time. The fine-grained neural recording and phoneme-level transcription provided in the T15 data set is ideal for this type of analysis, and the Brain2Text inference pipeline could be extended to produce outputs as probability distributions across phonemes for each frame, rather than the compact final output, allowing the neuroscientist to observe the temporal dynamics of decoding.

### **10.4 Human-Computer Interaction Without Motor Input**

The approaches employed in Brain2Text are applicable to a larger question in HCI: How can the computer be controlled by someone without a motor channel? In recent years, gaze, EMG and EEG have been some alternatives to the mouse or keyboard investigated within the frame of HCI. The most bandwidth rich non-motor input channel is intracortical BCI, but also the most difficult to implement (surgery). For those who still have some motor control, but find traditional input devices fatiguing and/or unreliable, non-invasive options such as EEG-based command interfaces offer a good compromise.

Brain2Text's take on this space is largely the deployment structure. The pattern of using REST API and React as the frontend could be used with any BCI backend, not only with GRU-based neural decoding. A developer creating an EEG-based command interface would be able to use the same front end and replace the inference module for the back end.

### **10.5 Educational and Outreach Value**

Many people have no clear understanding of what a brain-computer interface is. The term is used, "mind reading," which is inaccurate and disturbing. The best way to clarify the process,

rather than by using words, is to show, concretely, that it involves a trained machine learning model reading electrical activity from particular electrodes in the motor cortex, and mapping that activity to the probabilities of phonemes.

This is why the demo version of Brain2Text was created. It can be demonstrated on any computer with a web browser (no hardware required), it requires no internet access, and it visualizes the four-step process (acquisition, feature extraction, phoneme decoding, word lookup) and timing them. Such an explanatory approach is actually useful for high school and undergraduate students, for medical students studying diseases of the nervous system and for physicians considering implementing BCI for an individual patient.

## **10.6 Integration with Larger Language Models**

The output from the current Brain2Text is a short phrase; usually the first 5 to 10 words of the trial are used. This is enough for simple communication of needs, not for conversation. A major application of the system that would be very useful is to apply a large language model as a post processor.

This output could be sent to an LLM along with a system prompt stating: “User has ALS and is using a brain computer interface. The following is an utterance which may have errors and is phoneme-decoded. The following encoding would make it possible for the LLM to leverage the conversational context to resolve some decoding ambiguities that the phoneme-to-text module fails to handle and, for instance, decide between “water” and “waiter”, depending on whether the conversation concerns beverages or restaurants.

Metzger et al. [2] showed how this can be accomplished at the word level using a large vocabulary neural decoder. The benefit of the phoneme-level approach of Brain2Text is that it may not be vocabulary limited, but it can be more complex for the LLM to do the linguistic reconstruction. This is an open empirical question to be investigated in future research: does the phoneme-level input result in improved LLM-corrected output as compared to the word-level input.

## 11 Conclusion and Future Scope

### 11.1 Conclusion

As part of the Brain2Text project, the team aimed to create a fully functional brain-to-text communication system that was based on the Brain-to-Text Challenge 2025 data and accessible via a contemporary web interface. The following goals were met after working through the entire stack from loading intracortical neural recordings, to rendering the decoded text in the browser.

A five-layer GRU model was trained on the T15 CopyTask benchmark, and trained using CTC loss, and it was packaged as a portable checkpoint and integrated into a Python inference pipeline. It is variable-length input pipeline, uses greedy CTC decoding to recover strings of ARPAbet phonemes, and translates the ARPAbet phonemes to English words using CMU dictionary with a frequency ordering; has LLM fall-back. The 5 Flask API endpoints behaved as expected and had the same latency of less than 200ms when performing inference on consumer CPU hardware for both multipart file upload and JSON modes.

The aim of making the system accessible to a non-specialist audience was obtained with the React frontend. The demo mode allowed a user to use the application to develop a believable presentation without the need for running a backend, having a trained model or needing Internet connection. The live mode showed that the whole pipeline from uploading the .npy file to getting the decoded sentence works within the response time of an interactive interface.

On a larger scale, this project shows that it is not limited to a specialist's Jupyter notebook to build a functional neural speech decoding system. We've wrapped it in a documented REST API and an easy-to-use web GUI and created a product that we are able to give to a clinical researcher and tell them "Look, you're about to give this to a software engineer, you're not going to let them down, right?".

### 11.2 Future Scope

There are several directions in which the Brain2Text system is meaningful to extend and improve:

**Transformer-based decoders:** The GRU architecture is a sequential architecture, and cannot directly attend to long-range temporal dependencies. The decoding accuracy would likely get further increased if the GRU was replaced by the Transformer with the same variable-length input, and if the rotary positional embeddings were applied, the accuracy could be further

increased, particularly for longer phrases, for which the phoneme context window is more relevant.

**Language model integration:** Currently, the greedy CTC decoder lacks any language model knowledge of the frequency of English words or sentences. A character-level or word-level language model would be helpful to significantly reduce the word error rate by assigning a negative score to a phonetically likely but linguistically unlikely sequence of words, as part of the beam search decoder.

**Real-time streaming:** In the present system one complete trial is processed at a time, known as real-time streaming. If it is an implementation of streaming with WebSockets, the partial decoding may be displayed in real time as it occurs, so that the latency felt by the user for an assistive real-time communication, is more acceptable..

**Smaller and more limited datasets:** Smaller and limited datasets: The T15 CopyTask dataset has data for only one participant and few copy-task phrases. If a system is to be trained to perform outside the specific motor movements and electrode configuration of one person, then it must be trained using multi-participant data sets of naturalistic speech intent or simulated data sets created using a model of neural populations.

**Multilingual support:** CMU Pronouncing Dictionary has American English. If the phoneme to text module could be extended to other languages, the architecture would not have to change significantly since a new phoneme dictionary for the language (e.g. CMU Sphinx dictionaries for French, Spanish, or Mandarin) would be required along with training data.

**Edge deployment:** In Edge deployment, the deployed model can be reduced by quantisation (such as INT8 via ONNX Runtime) and/or knowledge distillation, and the system can be deployed directly to an embedded device on the bed side such as a Raspberry Pi or NVIDIA Jetson instead of using the network as is currently in the client-server approach.

**Clinical integration:** To be ready for production, a version of Brain2Text would need to be integrated with a production-grade real-time neural data acquisition system, undergo institutional data governance review, consider regulatory approval for a medical device, and undergo usability testing with target users. These steps are not part of the final year engineering project, rather they are logical and interesting further steps on the project.



## References

- [1] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson, and K. V. Shenoy, “High-performance brain-to-text communication via handwriting,” *Nature*, vol. 593, pp. 249–254, 2021.
- [2] S. L. Metzger, J. R. Liu, D. A. Moses, M. E. Dougherty, M. P. Seaton, K. T. Littlejohn, J. Chartier, G. K. Anumanchipalli, A. Tu-Chan, K. Ganguly, *et al.*, “A high-performance neuroprosthesis for speech decoding and avatar control,” *Nature*, vol. 620, pp. 1037–1046, 2023.
- [3] G. K. Anumanchipalli, J. Chartier, and E. F. Chang, “Speech synthesis from neural decoding of spoken sentences,” *Nature*, vol. 568, pp. 493–498, 2019.
- [4] D. A. Moses, S. L. Metzger, J. R. Liu, G. K. Anumanchipalli, J. G. Makin, P. F. Sun, J. Chartier, M. E. Dougherty, P. M. Liu, M. A. Berger, *et al.*, “Neuroprosthesis for decoding speech in a paralyzed person with anarthria,” *New England Journal of Medicine*, vol. 385, no. 3, pp. 217–227, 2021.
- [5] M. Nakanishi, Y. Wang, T.-P. Jung, J. K. Zao, Y.-Y. Chien, P. Diniz-Filho, T. Dau, S. J. David, R. Mallick, and G. Cauwenberghs, “Enhancing detection of ssveps for a high-speed brain speller using task-related component analysis,” *IEEE Transactions on Biomedical Engineering*, vol. 65, no. 1, pp. 104–112, 2018.
- [6] Z. Wang, X. Chen, and S. Gao, “Silent speech recognition from eeg using deep neural networks,” *Journal of Neural Engineering*, vol. 19, no. 4, p. 046011, 2022.
- [7] J. G. Makin, D. A. Moses, and E. F. Chang, “Machine translation of cortical activity to text with an encoder–decoder framework,” *Nature Neuroscience*, vol. 23, pp. 575–582, 2020.
- [8] A. Défossez, C. Caucheteux, J. Rapin, O. Kabeli, and J.-R. King, “Decoding speech perception from non-invasive brain recordings,” *Nature Machine Intelligence*, vol. 5, pp. 1097–1107, 2023.
- [9] J. Tang, A. LeBel, S. Jain, and A. G. Huth, “Semantic reconstruction of continuous language from non-invasive brain recordings,” *Nature Neuroscience*, vol. 26, pp. 858–866, 2023.
- [10] C. Herff, D. J. Krusienski, and P. Kubben, “The potential of stereotactic-eeg for brain-computer interfaces: current progress and future directions,” *Frontiers in Neuroscience*, vol. 14, p. 123, 2020.
- [11] M. A. Lebedev and M. A. L. Nicolelis, “Brain-machine interfaces: past, present and future,” *Trends in Neurosciences*, vol. 29, no. 9, pp. 536–546, 2006.

- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd International Conference on Machine Learning*, pp. 369–376, ACM, 2006.
- [14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [15] Kaggle BCI Team, “Brain-to-text benchmark ’25.” Kaggle Competition Dataset, 2025.
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [17] Pallets Team, “Flask: A lightweight WSGI web application framework.” Official Documentation, 2024.
- [18] S. Bird, E. Klein, and E. Loper, “Natural language processing with python.” O’Reilly Media, 2009.
- [19] Meta Open Source, “React – a JavaScript library for building user interfaces.” Official Documentation, 2024.
- [20] Evan You and contributors, “Vite: Next generation frontend tooling.” Official Documentation, 2024.

## A Appendix

### A.1 Project Repository Structure

The complete Brain2Text repository is organised as follows. The listing below reflects the state of the codebase at the time of submission.

```
Brain2Text/
+-- server/
|   +-- app.py                # Flask REST API (5 endpoints)
|   +-- inference.py          # GRU loader, run_inference()
|   +-- phoneme_to_text.py     # CMU dict + LLM + echo fallback
|   +-- model_GRU_pretrained.pkl # Trained checkpoint (~75 MB)
+-- frontend/
|   +-- src/
|   |   +-- App.jsx           # Root component, all state
|   |   +-- index.css         # Global styles
|   +-- index.html
|   +-- package.json          # React 19, Vite 8, lucide-react
|   +-- vite.config.js        # Dev server proxy config
+-- demo_data/
|   +-- trial_001.npy          # "open the door"
|   +-- trial_002.npy          # "water bottle"
|   +-- trial_003.npy          # "hello world"
|   +-- trial_004.npy          # "good morning"
|   +-- trial_005.npy          # "thank you"
|   +-- trial_006.npy          # "how are you"
|   +-- trial_007.npy          # "i want water"
|   +-- trial_008.npy          # "turn off light"
+-- start.sh                   # One-command local launcher
+-- colab_setup.py             # Colab + ngrok deployment
+-- requirements.txt           # Python dependencies
+-- README.md                  # Setup and usage documentation
```

## A.2 Python Dependency Versions

Table A.1 lists the pinned dependency versions used in the final deployment. These were chosen to avoid the NumPy 2.x / PyTorch 2.x incompatibility described in Section 4.7.

Table A.1: Python dependencies (requirements.txt)

| Package    | Version constraint |
|------------|--------------------|
| torch      | >=2.0.0            |
| numpy      | >=1.24.0, <2.0.0   |
| flask      | >=3.0.0            |
| flask-cors | >=4.0.0            |
| nltk       | >=3.8.0            |
| requests   | >=2.28.0           |

## A.3 ARPAbet Phoneme Vocabulary

Table A.2 lists the 40 ARPAbet phoneme symbols used as the model output vocabulary, grouped by category.

Table A.2: ARPAbet phoneme vocabulary (index 0 reserved for CTC blank)

| Category           | Symbols                                                    |
|--------------------|------------------------------------------------------------|
| Monophthong vowels | AA, AE, AH, AO, AW, AY, EH, ER, EY, IH, IY, OW, OY, UH, UW |
| Stops              | B, D, G, K, P, T                                           |
| Fricatives         | DH, F, S, SH, TH, V, Z, ZH                                 |
| Affricates         | CH, JH                                                     |
| Nasals             | M, N, NG                                                   |
| Approximants       | L, R, W, Y                                                 |
| Aspirate           | HH                                                         |
| <b>Total</b>       | 40 phonemes + 1 CTC blank = 41 output classes              |

## A.4 Flask API Request and Response Formats

**POST /api/predict (file upload):**

Request:

Content-Type: multipart/form-data

Body: file=<.npz binary>

Response:

```
{
  "phonemes": "HH AH L OW | W ER L D",
  "text": "hello world",
  "inference_time_ms": 112.4,
  "sequence_length": 98,
  "model_type": "GRU",
  "status": "success"
}
```

**GET /api/health:**

Response:

```
{
  "status": "ok",
  "model_loaded": true,
  "model_type": "GRU",
  "layers": 5,
  "hidden_size": 768,
  "best_per": 28.4,
  "vocab_size": 41
}
```

**POST /api/phonemes-to-text:**

Request:

Content-Type: application/json

Body: {"phonemes": "TH AE NG K | Y UW"}

Response:

```
{  
  "text": "thank you",  
  "method": "cmu_dict",  
  "status": "success"  
}
```

## A.5 Demo Trial Phoneme Reference

Table A.3 provides the complete ARPAbet representation for all eight demo trials, included here as a reference for anyone extending the demo data.

Table A.3: Complete demo trial phoneme reference

| Phrase         | ARPAbet sequence (pipe = word boundary) |
|----------------|-----------------------------------------|
| open the door  | OW P AH N   DH AH   D AO R              |
| water bottle   | W AO T ER   B AA T AH L                 |
| hello world    | HH AH L OW   W ER L D                   |
| good morning   | G UH D   M AO R N IH NG                 |
| thank you      | TH AE NG K   Y UW                       |
| how are you    | HH AW   AA R   Y UW                     |
| i want water   | AY   W AA N T   W AO T ER               |
| turn off light | T ER N   AO F   L AY T                  |

## A.6 Startup Script Reference

The `start.sh` script is reproduced below for reference. It is a POSIX-compatible bash script and runs on macOS and Linux without modification.

```
#!/bin/bash  
  
echo "Starting Brain2Text backend..."  
cd server  
python app.py &  
BACKEND_PID=$!  
cd ..  
  
sleep 2
```

```
echo "Starting Brain2Text frontend..."
cd frontend
npm run dev &
FRONTEND_PID=$!
cd ..

echo "Brain2Text running:"
echo "  Backend : http://localhost:5050"
echo "  Frontend: http://localhost:5173"
echo ""
echo "Press Ctrl+C to stop both services."

trap "kill $BACKEND_PID $FRONTEND_PID 2>/dev/null; exit" SIGINT SIGTERM
wait
```

ayush

# Brain2Text\_\_Neural\_Signal\_Decoding\_to\_Text\_via\_GRU\_ba...

 D Y Patil International University, Akurdi

## Document Details

Submission ID

trn:oid::3618:139068257

Submission Date

May 15, 2026, 11:36 AM GMT+5:30

Download Date

May 15, 2026, 11:49 AM GMT+5:30

File Name

Brain2Text\_\_Neural\_Signal\_Decoding\_to\_Text\_via\_GRU\_based\_Deep\_Learning\_and\_Web\_Deploym....pdf

File Size

287.0 KB

63 Pages

17,603 Words

93,158 Characters



## \*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

# ayush

## Brain2Text\_\_Neural\_Signal\_Decoding\_to\_Text\_via\_GRU\_ba...

 D Y Patil International University, Akurdi

### Document Details

Submission ID

trn:oid::3618:139068257

Submission Date

May 15, 2026, 11:36 AM GMT+5:30

Download Date

May 15, 2026, 11:49 AM GMT+5:30

File Name

Brain2Text\_\_Neural\_Signal\_Decoding\_to\_Text\_via\_GRU\_based\_Deep\_Learning\_and\_Web\_Deploym....pdf

File Size

287.0 KB

63 Pages

17,603 Words

93,158 Characters

# 5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

## Match Groups

-  **79 Not Cited or Quoted 5%**  
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**  
Matches that are still very similar to source material
-  **0 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

## Top Sources

- 3%  Internet sources
- 2%  Publications
- 5%  Submitted works (Student Papers)

## Integrity Flags

### 0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.